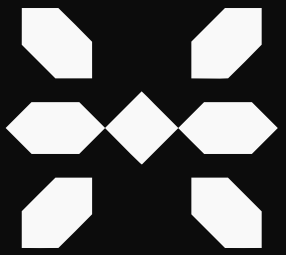
HOOX · AXIS · DOCUMENTATION



TRADERS · RESEARCHERS · SELF-HOSTERS

END USER MANUAL

Install the PWA, compose plugins, research workflows

Open charting PWA — own the axes, swap the engine

2026-11-01 DIN A4 · HTTPS://HOOX.SH/AXIS/DOCS OPEN SOURCE

CONTENTS

01 Overview

02 Overview

03 Installation

04 Quick Start

05 Compose Recipes

06 Manager And Settings

07 Pyne Agent

08 Data Source Manager

09 On Chain

10 Script Library

11 Builtin Scripts

12 Strategy And Results

13 Hyperparameter Optimisation

14 Drawings

15 Alerts

16 Mcp

17 Screenshots

18 Troubleshooting

19 Glossary

20 Faq

OVERVIEW

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "AXIS Documentation" description: "Open charting PWA — orthogonal sources, streams, engines, and storage. Own the axes; swap the engine."

AXIS DOCUMENTATION

AXIS is an installable charting PWA (v**2.27.0**, `package.json` / `VERSION`) that treats **price**, **time**, and **calculation** as separable axes. Release history is `CHANGELOG.md` (online changelog). Sources load history. Streams advance the present. Engines evaluate Pine Script™ via **PYNE**. Storage keeps your script library. Compose any lawful combination without a proprietary charting host.

AXIS workspace with BTCUSDT daily candles, watchlist, volume, RSI, and Pine editor

Pine Script™ and TradingView® are trademarks of TradingView®, Inc. Cloudflare® is a registered trademark of Cloudflare®, Inc. This project is independent and not affiliated with or endorsed by TradingView®, Inc. or Cloudflare®, Inc.

What's new

The current **2.27.0** build — full detail in the changelog:

- **Pre-eval call scanning is O(n)** — idle lint no longer re-scans from offset 0 on every `(`; open parens stack and each call parses when its closer lands. Measured ~5.9 s → 43 ms at 2 000 lines (and ~22 s → 120 ms at 4 000) with both lint passes on. Unclosed mid-edit calls are skipped.
- **Offline Pine symbol navigation** — local definition / references / rename / occurrence highlight (`F12`, `Shift-F12`, `F2`, automatic highlight) from one editor symbol index. Works with Pyodide and offline — no Pro API round trip. Also in the command palette.
- **Offline PYNE lint parity: `W002` and `W104`** — pre-v5 `//@version` and v6-removed kwargs (`when=` on `strategy.*` orders; `transp=` on drawing / plot calls) warn locally. Settings → Editor toggles; Pyodide / offline match `POST /lsp/diagnostics`.
- **Vendored browser wheel is `pynescript-0.6.29`** — Pyodide `/vendor/` path matches the shipped wheel (product PYNE may be ahead; document the vendored wheel). See engine plugins.
- **2.26 tip-only ranging** — while panning/zooming, overlay / bgcolor / OHLC paints stay tip-`update` only; full-series `setData` and marker rebuilds defer until the gesture ends. Cuts pan-frame spikes under live ticks. See the changelog for **2.26.0** and **2.27.0**.

Abstract

AXIS is a charting PWA, not a language rewrite. Language fidelity lives in PYNE. The PWA contributes:

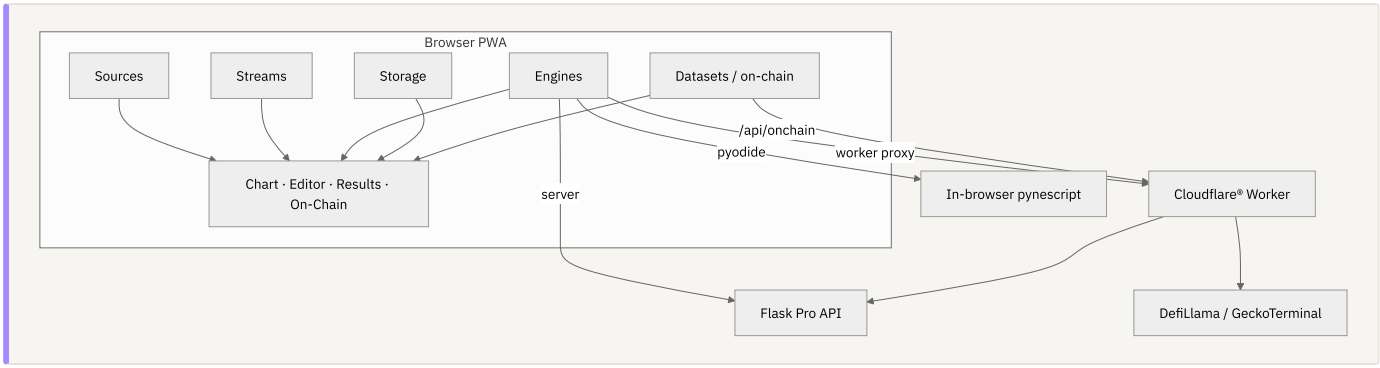
1. A **unified plugin registry** (`source | stream | engine | storage | dataset | component`)
2. A **Solid + Vite** product UI (chart, editor, results, manager, Workers Manager, on-chain panel) with a **mobile-first responsive shell** and **global keyboard shortcuts** (recordable, persisted)
3. Optional backends: local Flask, offline Pyodide, Cloudflare® Worker (+ DO/KV/D1/R2 + `/api/onchain` proxy)
4. **AXIS CLI** + optional **Tauri** desktop shell for operators
5. **MCP** on the Worker (`POST /mcp`) so agents can drive APIs and a connected tab — MCP (agents)

Invariant (AXIS ≠ engine): the shell never embeds a closed interpreter. Evaluation is always an engine plugin.

Product map

Track	Role	Start here
End User	Install, compose recipes, research workflows	End User hub
Architecture	ADRs, topologies, state namespaces	Architecture
Plugins	Contracts and built-in catalogs	Plugins
UI	Chart, editor, panels, store	UI
Worker	CF Pages + Worker data plane	Worker
DevOps	Build, CLI, VPS, CORS, CI	DevOps · AXIS CLI
Evaluation	Flask / Pyodide / Worker proxy — not PyneTS, not HOOX execution	Evaluation map
PYNE (separate)	Grammar & evaluator	PYNE docs

Conceptual model



Formal active set:

```
Active set = { source, stream, engine, storage }
Contract namespace: pynescript.axis.plugins.v1
App state: pynescript.axis.v1
On-chain dataset plugin: defillama-tvl only. Pool candles are source geckoterminal-ohlcv, not a dat
```

Compose in one glance

Recipe	Source	Stream	Engine	Storage
Offline lab	`mock-walk`	`mock-poll`	`pyodide`	`local`
Desk research	`binance-rest`	`binance-ws`	`server` → Flask	`local`
AXIS at the edge	venue REST	venue WS / DO	`server` → Worker	`cloud`
On-chain TVL + CEX	`binance-rest` + DefiLlama dataset	venue WS	any	`local`
DEX pool candles	`geckoterminal-ohlcv`	`mock-poll`	any	`local`
Repo library	any	any	any	`git`

Full recipes: Compose recipes.

Infrastructure names (do not rename lightly)

Surface	Value
CF Pages project	`axis`
Pages URL	`https://axis.hoox.sh`
CF Worker script	`worker-axis`
Worker URL	`https://worker.axis.hoox.sh`
Health JSON service	`worker-axis`
Product brand	**AXIS**

Demo topologies

- Dev: Vite `:3000` + Flask `:5002` (`axis install` then `bun run dev`)

- Desktop: `bun run desktop:dev` (Tauri 2)
- Static: `bun run build` + `axis_pwa_server.py` `:8081`
- Worker: `axis dev worker` / wrangler on `:8787`
- Prod: `axis deploy` → Worker `worker-axis` (`https://worker.axis.hoox.sh`) + Pages `https://axis.hoox.sh`

See topologies, AXIS CLI, and local dev.

Offline & agent exports

`llm.txt`, `llms.txt`, and the track PDFs are generated outside this package. AXIS `package.json` has no `docs:exports` script. The generator is `node scripts/docs/generate-exports.mjs` in the PYNE repo, and the landing site copies these pages when it builds.

Kind	Path
Full-corpus LLM pack	`llm.txt`
LLM site map (llmstxt.org)	`llms.txt`
Track PDFs (A4)	`/exports/axis-*-manual.pdf`

See also

- Quick start
- Plugin contracts
- ADRs
- PYNE runtime

OVERVIEW

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "End User" description: "Install AXIS, compose source/stream/engine/storage recipes, and run research workflows without a proprietary chart host."

END USER

AXIS is an installable charting PWA (product **2.27.0**): load history, stream the present, evaluate Pine Script™ through a pluggable engine, and keep a script library on disk, IndexedDB, cloud, or git. This track is for operators and researchers who use the AXIS—not for plugin authors or Worker deployers. Current-build highlights live on the docs home; full release notes in the changelog.

AXIS PWA shell: topbar, watchlist, chart panes, and editor

Language fidelity (grammar, builtins, runtime semantics) lives in PYNE. AXIS never reimplements the language; it **hosts** evaluation via engine plugins.

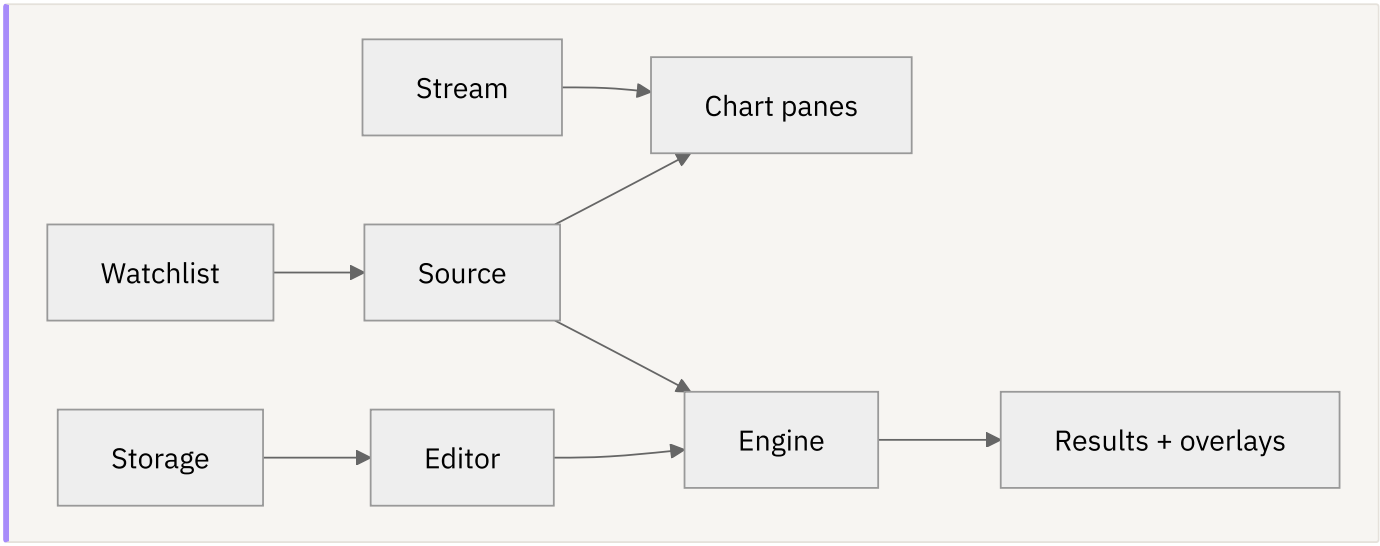
Abstract

You compose four axes:

Axis	Question it answers
Source	Where do historical OHLCV bars come from? (CEX venues or DEX pools)
Stream	How does the live bar advance?
Engine	Who evaluates the script?
Storage	Where do saved scripts live?
On-chain	Protocol TVL, DEX OHLCV, events — parallel plane (not a wallet)

Any lawful combination is valid. Offline lab, desk research against Binance + Flask, AXIS at the edge through a Cloudflare® Worker, and a git-backed library are all first-class recipes—see Compose recipes.

Conceptual model



The shell (topbar, chart, editor, results, manager) is **AXIS**. Calculation is always `engine.run({ script, bars, config })`. Persistence of scripts is always a storage plugin.

Interface surface

Surface	Role
Topbar	Symbol, interval, source/stream/engine pickers, Load, Run, Live, On-Chain
Watchlist	Quick symbol switch + quote poll
Chart	lightweight-charts panes, drawings, trade markers, on-chain overlays, price-scale decimals
Editor	CodeMirror 6 Pine Script™ document (docked / popout)
Results	Events, Strategy, Optimise, Plots, Metrics, Raw + export
On-Chain	DefiLlama TVL, GeckoTerminal DEX, spike events, export / refresh
Manager	Plugin catalog, URL install (incl. **component** / PYNE Agent), Script Library
Workers Manager	Backend health probes, presets, install helpers
Settings	Endpoint, engine, storage, on-chain proxy notes, MCP bridge
MCP	Agents on `POST /mcp` — Worker APIs + this tab (guide)
Desktop	Optional Tauri 2 shell (`bun run desktop:dev`)

Getting started

1. Installation — AXIS CLI, Vite dev, desktop, static `dist/`, or hosted PWA
2. Quick start — first load + first Run (`//@version=6`)
3. Compose recipes — offline, desk, multi-venue, CSV, edge, git

Guides

- Manager and settings — Plugins + **Workers Manager**
- PYNE Agent — natural language → scripts
- On-Chain data — TVL, DEX pools, events, Worker proxy
- Data Source Manager
- Script library — incl. v6 starter pack import
- Built-in scripts — original AXIS studies (Scripts → Built-in)
- Strategy and results
- Drawings
- Alerts — local price / Pine / on-chain; not HOOX orders
- MCP (agents) — Claude / Cursor / Inspector on `POST /mcp`
- Troubleshooting

Reference

- Glossary
- FAQ

Invariants (operator-facing)

1. **AXIS ≠ engine** — switching engine does not change the chart library; switching source does not change the language runtime.
2. **Active set is four-tuple** — source, stream, engine, storage are selected independently (with sensible defaults when source implies stream).
3. **API keys stay in the browser** — cloud storage and Pro keys are not uploaded by the shell except as request headers you configure.
4. **Chart OHLCV is ephemeral** — reload re-fetches for the topbar path; the **Data Source Manager** keeps a durable IDB `bars-cache` for deep history jobs.
5. Layout, script draft, and drawings survive in `pynescript.axis.v1`.

See also

- Architecture — ADRs and topologies
- Evaluation map — Flask / Pyodide / Worker / not PyneTS
- AXIS CLI — install / doctor / deploy
- UI — chart, editor, store internals
- AXIS and HOOX
- Plugins — contracts and catalogs
- PYNE runtime — evaluation semantics

INSTALLATION

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Installation" description: "Install AXIS via the AXIS CLI, Vite+Solid dev, static dist PWA, desktop shell, or Cloudflare® Worker backends."

INSTALLATION

AXIS ships as the **`axis`** package (Solid + Vite), product **2.27.0**. The product path is the UI under `src/` — use `bun run dev` or the built `dist/`. The pre-Solid static shell (`main.js`, root `style.css`) was removed in 2.21.0. The AXIS CLI package (`@hoox-sh/axis-cli` **0.3.6**) versions independently; its default PYNE Pro API URL is `https://pynescript.online` (override with `AXIS_PYNE_URL`).

AXIS CLI help: install, doctor, setup, deploy, health

Abstract

Operator modes:

Mode	Command surface	When
AXIS CLI (recommended ops)	`bun run axis` · `make axis-*`	Install / doctor / setup / deploy / health
Dev AXIS	`bun run dev` · `axis dev`	Day-to-day UI work
Desktop shell	`bun run desktop:dev`	Tauri 2 native window
Static PWA	`dist/` + `axis_pwa_server.py` `:8081`	Offline demo / VPS
AXIS at the edge	`axis deploy worker` (+ optional Pages)	Production data plane

Backend is optional when **Engine = Client-Side (Pyodide)** and sources/streams are mock or CSV.

Prerequisites

- **Bun ≥ 1.2** for package install, Vite, and the AXIS CLI
- **Python 3.10+** if you use Flask Pro API (`make run`) or `axis_pwa_server.py`
- Modern Chromium / Firefox / Safari (PWA install works best on Chromium)
- Optional: Cloudflare® auth (`CLOUDFLARE_API_TOKEN` or `wrangler login`) for Worker deploy
- Optional desktop: Rust + platform webview libs — Desktop (Tauri)

CLI-first bootstrap (recommended)

```
# From the AXIS monorepo root
bun install
cd packages/cli && bun install && cd ../..

axis install          # app + worker/ + CLI deps
axis doctor           # toolchain + optional CF auth; wrangler.toml warns until `axis setup`
axis setup            # ensure wrangler.toml + local D1 schema
# repo aliases (no global install): bun run axis:install / axis:doctor / axis:setup
# or: make axis-install && make axis-doctor && make axis-setup
```

Command	Repo alias (no global install)
`axis`	`bun run axis`
`axis install`	`bun run axis:install`
`axis doctor`	`bun run axis:doctor`
`axis setup`	`bun run axis:setup`
`axis deploy worker`	`bun run axis:deploy`
`axis health`	`bun run axis:health`
`axis …`	`make axis ARGS="…"` pass-through
`axis …`	`make axis-install` · `axis-doctor` · `axis-setup` · `axis-deploy` · `axis-health`

Full command surface: AXIS CLI.

Dev AXIS

```
# Terminal 1 — Pro API (server engine) from sister pyne repo
# Clone https://github.com/hoox-sh/pyne as ../pyne (local dir is sometimes still named pynescript)
make -C ../pyne run          # Flask :5002

# Terminal 2 — AXIS
bun run dev                  # Vite :3000
# or: bun run axis dev
```

Open `http://localhost:3000`. Default symbol `BTCUSDT`, engine often `server` with endpoint `http://localhost:5002` (or demo host `https://axis.hoox.sh`).

Desktop shell (optional)

```
bun run desktop:dev          # Tauri window + Vite HMR
bun run desktop:build        # native installers
```

See Desktop (Tauri).

Edge Worker (local)

```
bun run axis dev worker      # wrangler :8787
# or: cd worker && bun run dev
```

In AXIS: open **Workers Manager** (topbar activity icon) or Settings → set **Backend URL** to `http://127.0.0.1:8787`. Production Worker is `worker-axis` at `https://worker.axis.hoox.sh` — see topologies.

Deploy checklist (prod)

```
axis setup --github-client-id Ov23li… --remote-d1
axis secret put ADMIN_TOKEN
axis secret put EXTERNAL_BACKEND
axis deploy
axis health --oauth
```

Production build (static)

```
axis install
bun run build                # → dist/
python3 axis_pwa_server.py   # serves dist/ on :8081
```

Confirm:

- App shell loads; chart requests history
- DevTools → Application → Manifest + Service Worker
- Icons 192/512 from `public/assets/`

Pyodide assets and vendor wheels under `public/pyodide/` and `public/vendor/` must be present in `dist/` for offline engine— `bun run build` copies them via Vite `public/`. The **2.27.0** shipped vendored wheel is `pynescript-0.6.29-py3-none-any.whl` (product PYNE may be ahead of the wheel; document the filename under `/vendor/`).

Offline-first lab (no Flask)

1. Source → **Mock Walk**
2. Stream → **Mock Poll** (or **None**)
3. Engine → **Client-Side (Pyodide)**
4. Storage → **Local**

Disable network in DevTools; **Run** still executes. First Pyodide boot downloads/loads self-hosted runtime from the origin—allow that once while online, then go offline.

CORS when using server engine

Browser origin → Pro API must allow your AXIS origin. Flask uses `ALLOWED_ORIGINS`. Localhost regex is typically included; for a VPS demo host, set an explicit origin list. Symptom of failure: `POST /run` blocked in Network tab (no `Access-Control-Allow-Origin`). Worker CORS: CORS and origins.

PWA install

- Manifest: void theme `#0a0b10`, name AXIS
- Service Worker: cache-first shell; network-first `/api/*`; offline API returns structured failure so Pyodide path remains usable
- Chrome/Edge: install icon in the omnibox; AXIS also shows a modest **Install app** chip in the topbar when the browser fires `beforeinstallprompt` (right-click hides it for the session)

Verification checklist

Check	Expect
Load	Bars on chart for default symbol
Topbar	Source / Stream / Engine pickers populated
Run (server)	Flask or Worker responds; plots overlay
Run (pyodide)	Offline OK after warm-up
Manager	Catalog lists built-ins
Workers Manager	Health cards for local/prod backends
Theme	Dark/light toggle persists
`axis doctor`	Required toolchain green. Missing `worker/wrangler.toml` is a warning on a fresh clone until `axis setup` (copies `wrangler.toml.example`).

Failure modes

Symptom	Likely cause	Fix
Empty chart	Source network / CORS / wrong symbol	Load again; try mock-walk
Engine errors immediately	Endpoint down or wrong URL	Settings / Workers Manager → Probe
Pyodide “BadZipFile” / HTML	SPA fallback instead of wheels	Ensure `public/vendor` and pyodide in `dist/`
SW stale UI	Aggressive cache	Unregister SW or hard reload
`API_KEYS_REQUIRED` on scripts	D1 without KV	Bind `API_KEYS` or local `ALLOW_OPEN_KEYS=1` — Auth

Internals (repo paths)

Path	Role
`src/index.tsx`, `src/app.tsx`	Solid entry
`vite.config.ts`	Build
`axis_pwa_server.py`	Static host
`public/manifest.webmanifest`, SW	PWA
`worker/`	Cloudflare® Worker data plane
`packages/cli/`	AXIS CLI (`@hoox-sh/axis-cli`)
`src-tauri/`	Desktop shell

See also

- Quick start
- Compose recipes
- AXIS CLI
- DevOps local dev
- Desktop (Tauri)
- Worker

QUICK START

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Quick start" description: "First historical load, first Pine Run, live stream, and reading the Results drawer in AXIS."

QUICK START

Fifteen-minute path from empty AXIS to plots, strategy stats, and optional live updates.

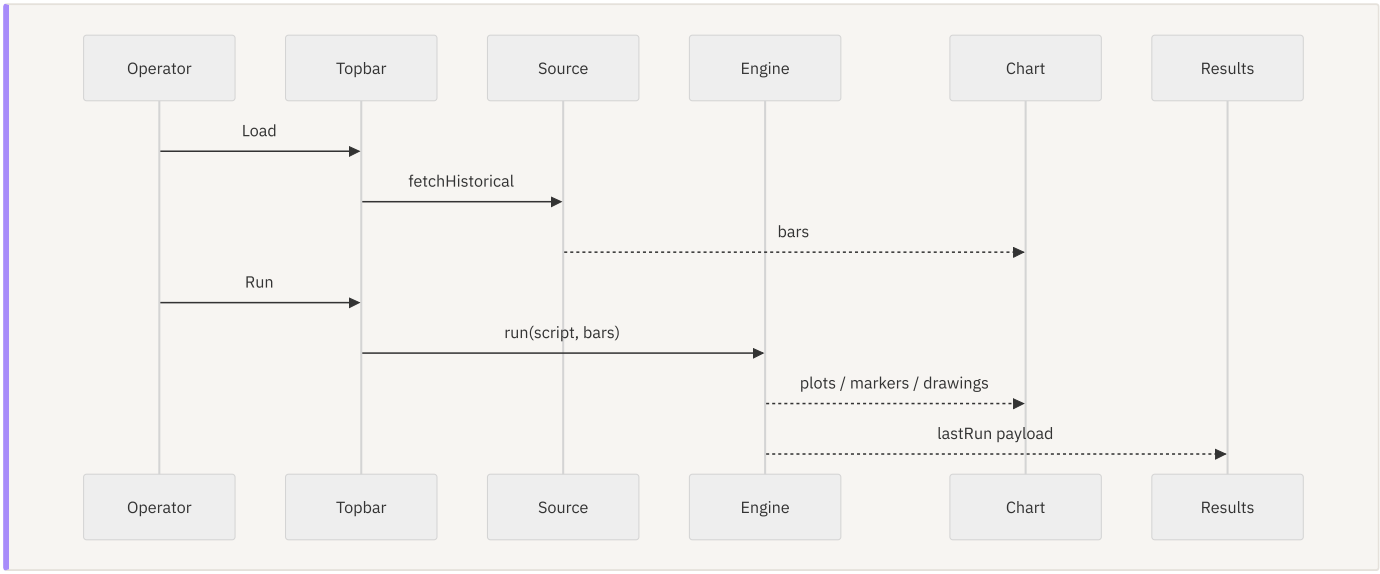
AXIS is a charting app you run in the browser (or install as a PWA / desktop app). You pick **where candles come from**, **whether they keep updating**, **who runs your Pine Script™**, and **where scripts are saved**. The status bar uses short chips for those four planes:

Chip	Means
SRC	**Source** — historical candles (Binance REST, CSV, mock, …)
STR	**Stream** — live ticks while **Live** is on (WebSocket or mock poll)
ENG	**Engine** — who calculates (this tab / a local server / a Worker)
STO	**Storage** — where scripts live (this browser, cloud, or git)

Abstract

1. Ensure AXIS is running (Installation).
2. Load bars for a symbol.
3. Paste or write a minimal indicator / strategy.
4. **Run** → inspect chart overlays + Results.
5. Optionally enable **Live**.

Conceptual model



Interface surface

1. Load history

Topbar fields that matter:

Control	Default-ish	Notes
Symbol	`BTCUSDT`	Venue-specific for `binance-rest`
Interval	`1d`	Watchlist + chart share interval vocabulary
Source	`binance-rest`	Or mock / CSV
Stream	`binance-ws`	Paused until Live
Engine	`server` or `pyodide`	Settings for endpoint

Click **Load** (or pick a watchlist row). Status bar should move through loading → ready with bar count context.

AXIS after Load: BTCUSDT candles, volume, and RSI pane Load history then Run an RSI script

CSV path: Source = **CSV upload** → pick file when prompted → bars inject via upload store.

Deep history: for multi-page backfill to a past date (background, with gap validation), open the **Data** panel — see Data Source Manager.

2. Minimal script

Open the editor (docked right by default). Example indicator:

```
//@version=6
indicator("AXIS smoke", overlay=true)
plot(close, "close")
plot(ta.sma(close, 20), "sma20")
```

Strategy smoke (events for Results → Strategy):

```
//@version=6
strategy("AXIS strat smoke", overlay=true)
longCond = ta.crossover(ta.sma(close, 10), ta.sma(close, 30))
if longCond
    strategy.entry("L", strategy.long)
if ta.crossunder(ta.sma(close, 10), ta.sma(close, 30))
    strategy.close("L")
```

Language semantics: PYNE runtime.

3. Run

Run (topbar or editor). Pipeline:

1. Active engine `run({ script, bars, config })`
2. `lastRun` stored in memory (not fully persisted)
3. Results drawer opens
4. Chart applies overlay lines, trade markers, Pine drawings

Success: status “Completed in Nms”; plots on price pane (or dedicated indicator pane if non-overlay).

4. Read Results

Results Strategy tab after an SMA Cross run: stats, equity, closed trades

Tabs:

Tab	Content
Events	Normalized entry/exit/order stream
Strategy	Closed trades + win rate, PF, max DD
Plots	Series names, point counts, last value
Metrics	Runtime, engine, bar count, errors
Raw	Full JSON payload

Export: JSON run dump; trades CSV. Click a trade to scroll the chart to entry/exit time.

5. Live (optional)

Toggle **Live**. A stream plugin pushes bars; AXIS may re-run silently when `needsRerun` is set. Live off (`stream: null` , not a `none` plugin) freezes time. Mock poll works offline after engine warm-up.

6. Save a script (optional)

Manager → **Script Library** → name → Save. Backend is active storage (`local` default = IndexedDB). See Script library. Editor **Pull / Push** uses the same storage when configured for git/cloud.

7. Power tools (optional)

Tool	Where	Purpose
⌘/Ctrl+K	Global	Command palette — panels, Run, layout, editor toggles
F12 / Shift-F12 / F2	Editor focus	Go to definition, find references, rename (offline / Pyodide — no Pro API)
Screenshot	Module bar camera (before Studio)	Capture panes to PNG — drawings, watermark, 1×/2×, copy or download
Workers Manager	Topbar activity · ⌘K	Backend health, presets, PYNE Agent install
Layouts	Topbar	Multi-chart 1 / 2H / 2V / 4 + recipes
Replay	Topbar	Scrub history (full bars, cursor on last)
Compare	Topbar	Second symbol overlay
Price decimals	Price-pane scale	Auto or 0–8 — Charting
Debug / Pins	Editor header	Line chips vs chart markers — Debugging
Layers / Alerts	Panel toggles	Drawings list; local price alerts
Built-in studies	Scripts → **Built-in**, Add ▾, or ⌘K	Original AXIS RSI / MACD / SMA / …
v6 starter pack	Script Library → Import	`examples/script-library-starter.json`

Worked example: offline lab

Setting	Value
Source	mock-walk
Stream	mock-poll
Engine	pyodide
Storage	local

Load → Run smoke indicator → toggle Live. No Flask required.

Worked example: desk + Flask

Setting	Value
Source	binance-rest
Stream	binance-ws
Engine	server
Endpoint	`http://localhost:5002`
Storage	local

`make run` in repo root; Vite or static AXIS as installed.

Invariants

- Empty editor **Run** is a no-op.
- Errors set status `error` and still populate Results Metrics/Raw when payload exists.
- Switching source auto-aligns default stream (e.g. mock-walk → mock-poll).

Failure modes

Symptom	Action
CORS on `/run`	Fix `ALLOWED_ORIGINS` / use Pyodide
No trades in Strategy	Need entry+exit pair events; open strategy script
Live no updates	Turn Live on and pick a stream (`binance-ws` or `mock-poll`); check the network and the logs drawer
Pyodide first run slow	Expected warm-up; assets self-hosted under origin

See also

- Compose recipes
- Strategy and results
- Troubleshooting

COMPOSE RECIPES

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Compose recipes" description: "Lawful source × stream × engine × storage combinations: offline, desk, multi-venue, CSV, edge, and git library."

COMPOSE RECIPES

AXIS treats composition as the product. This page lists **recipes**—complete four-tuples with intent, constraints, and failure modes—not marketing tiers.

In the app: topbar **Studio** → **Wire** (or ⌘K → **Open Architecture**) opens the full-page compose studio. Start from a recipe, then swap or switch off any slot. Apply writes `source` / `live.streamId` / `engine` / `activePlugins` (and opens the on-chain panel when a dataset is selected). Empty slots link to **Plugins**; the active engine is configured on **Runtime**.

Studio Wire: Live Crypto recipe with Binance REST source selected

Abstract

Formal active set:

```
Active set = { source, stream, engine, storage }
```

Built-in ids (catalog may grow via URL plugins):

Kind	Built-ins
Source	`binance-rest`, `okx-rest`, `bybit-rest`, `coinbase-rest`, `kraken-rest`, `mexc-rest`, `geckoterminal-ohlcv`, `ccxt-rest`, `mock-walk`, `csv-upload`, `data-manager`, plus URL plugins
Stream	`binance-ws`, `okx-ws`, `bybit-ws`, `coinbase-ws`, `kraken-ws`, `mexc-ws`, `ccxt-ws`, `mock-poll`. There is no `none` stream. Live off is `stream: null` / `enabled: false`.
Engine	`server`, `pyne-worker`, `pyodide`
Storage	`local`, `cloud`, `git`

Recipe matrix

Recipe	Source	Stream	Engine	Storage	Needs net	Needs backend
Offline lab	mock-walk	mock-poll	pyodide	local	first boot only	no
Desk research	binance-rest	binance-ws	server → Flask	local	yes	Flask
Multi-venue desk	binance-rest + URL sources	binance-ws, or Live off	server	local	yes	Flask
CSV desk	csv-upload	Live off, or mock-poll	server or pyodide	local	no*	optional
AXIS at the edge	venue REST	venue WS / DO	server → Worker	cloud	yes	Worker (+ optional Flask proxy)
Repo library	any	any	any	git	yes (save/list)	git host API

*CSV needs local file; engine may still call network if `server`.

Offline lab

Intent: airplane mode, demos, CI-less teaching, pure AXIS UX.

Axis	Id	Why
Source	`mock-walk`	Synthetic OHLCV in-process
Stream	`mock-poll`	Synthetic bar advance
Engine	`pyodide`	In-browser `pynescript` via Pyodide
Storage	`local`	IndexedDB script library

Setup

1. Install static or dev AXIS with `public/pyodide` + `public/vendor` present (product **2.27.0** vendors `pynescript-0.6.29`).

2. Select the four-tuple in topbar / settings.
3. Load → Run while online once (warm Pyodide).
4. DevTools → Offline; re-run.

Invariants: no `POST /run`; no exchange traffic; drawings + library remain local.

Failure modes: missing wheels → HTML SPA fallback errors; cold start timeout on huge bar sets.

Desk research

Intent: real venue history + live klines + full PYNE server fidelity.

Axis	Id	Why
Source	`binance-rest`	Public REST klines
Stream	`binance-ws`	Public WS kline stream
Engine	`server`	Flask Pro API `POST {endpoint}/run`
Storage	`local`	Fast offline library

Setup

```
make run                # :5002
bun run dev
```

Settings → Backend URL `http://localhost:5002` → Test. Symbol like `BTCUSDT`, interval `1h` / `1d`.

CORS: AXIS origin must be allowed by Flask `ALLOWED_ORIGINS`.

Failure modes: rate limits; symbol not on venue; CORS; engine timeout on long history (AXIS scales timeout with bar count).

Multi-venue / multi-source

Intent: compare feeds without forking the AXIS.

Axis	Pattern
Source	Built-in `binance-rest` **or** Manager → Install URL plugin (e.g. CoinGecko example) → **Use**
Stream	Match the venue when it has a built-in stream. Otherwise leave Live off (`stream: null`).
Engine	Usually `server` for parity
Storage	`local` or `git` for shared research

Workflow

1. Manager → Catalog → capability badges (offline / auth / network / proxy).
2. **Use** sets active source.
3. Load and Run as usual.

Dynamic plugins rehydrate from localStorage install list on boot (`restoreInstalledPlugins`).

Failure modes: plugin URL not CORS-readable; invalid export shape; built-ins cannot be unregistered without force flags.

CSV desk

Intent: proprietary or offline CSV/OHLCV files.

Axis	Id
Source	`csv-upload`
Stream	Live off (typical) or `mock-poll` for artificial advance
Engine	`pyodide` (airgap) or `server`
Storage	`local`

Workflow

1. Source → CSV upload (file picker opens if no file yet).
2. AXIS parses via `parseOhlcvFile` into upload store.
3. Load injects bars; symbol field less critical for synthetic series.
4. Run strategies against that tape.

Invariants: bars stay in memory/session upload store—not the long-lived app state blob (size).

Failure modes: bad headers/time units; empty parse; re-select same file requires clearing input (AXIS resets file input after pick).

AXIS at the edge

Intent: browser AXIS + Cloudflare® Worker data plane (keys, usage, optional D1 scripts, DO fan-out).

Axis	Id	Why
Source	venue REST (built-in or proxy-aware)	History
Stream	venue WS **or** DO-backed example stream	Single upstream → N clients
Engine	`server` endpoint = Worker	Worker proxies `/api/run` → Flask or future in-worker runtime
Storage	`cloud`	`/api/scripts` + Bearer Pro key

Setup

1. `wrangler dev` / deploy Worker script **`worker-axis`** (`https://worker.axis.hoox.sh`).
2. Endpoint → Worker origin.
3. Storage cloud → API key from admin `/api/keys`.
4. Script Library uses cloud list/read/write; If-Match on concurrent writes.

Invariants: Pages project is `axis` (`https://axis.hoox.sh`); Worker script is `worker-axis` (`https://worker.axis.hoox.sh`); health JSON `service` is `worker-axis`.

Failure modes: missing Bearer key; D1 unbound → memory store (dev only); DO session query params wrong.

Git library

Intent: Pine sources as first-class repo files; Save = commit.

Axis	Id
Source / Stream / Engine	any research tuple
Storage	`git`

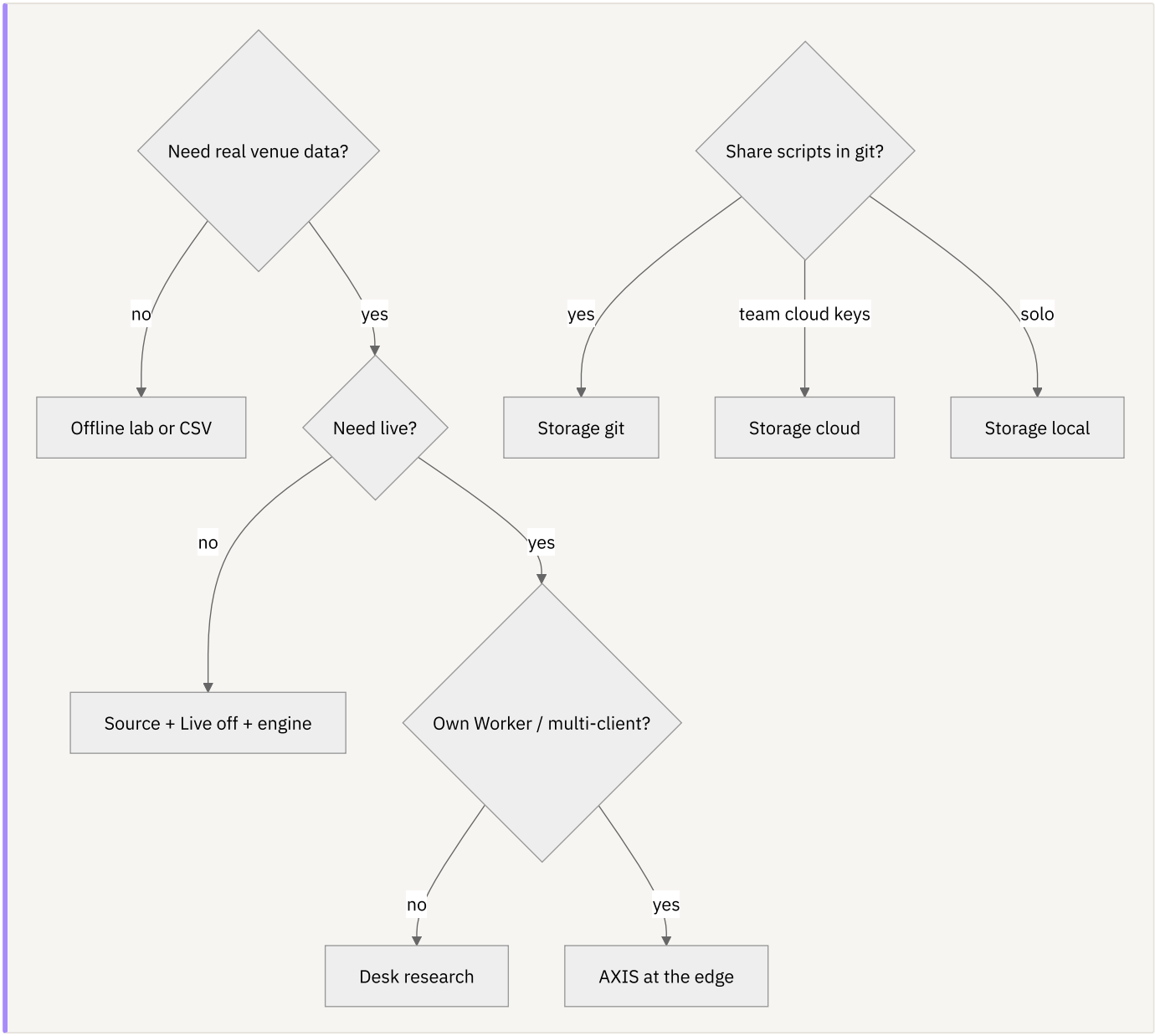
Config (Manager → Script Library or pluginsConfig)

Field	Notes
provider	`github` | `gitlab`
token	contents:write / api
owner, repo	GitHub path; GitLab projectId optional
branch	default `main`
basePath	default `pine-library`
apiBaseUrl	self-hosted forges

Invariants: drafts stay local; only explicit Save/remove hit the remote. Commit message template supports `{{name}}` / `{{iso}}`.

Failure modes: 401 token scope; wrong base path; GitLab path vs numeric id confusion.

Choosing a recipe



Internals

Concern	Path
Active selection	`src/store` `activePlugins`
Built-in catalogs	`sources/catalog.ts` , `streams/*` , `engines/catalog.ts` , `storage/catalog.ts`
URL plugins	`plugins/loader.ts`
Compose UI	`ui/Topbar.tsx` , `ui/PluginManager.tsx` , `ui/SettingsDialog.tsx`

See also

- Plugin contracts
- Manager and settings
- Topologies

- ADR-001 pluggable registry

MANAGER AND SETTINGS

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Manager and settings" description: "Runtime, Workers, Plugins, Wire, and Settings — how AXIS chrome is split."

MANAGER AND SETTINGS

Five studio pages share one full-viewport overlay. Runtime is the active calculation context; Workers and Plugins are catalogs, not tabs inside it.

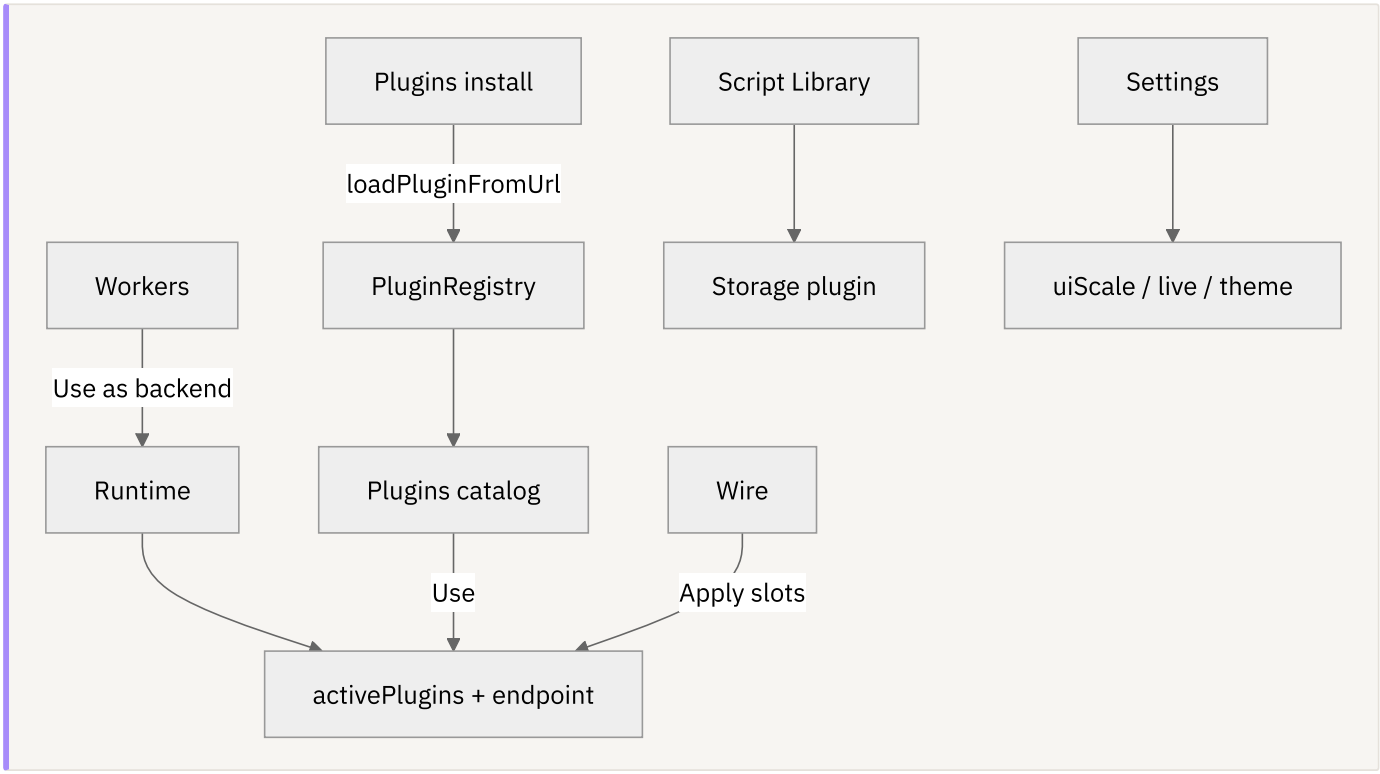
Studio Runtime: server engine, health, and backend endpoint
Studio rail switching Runtime, Wire, Settings, Workers, Plugins

Abstract

Surface	Opens from	Mutates
Runtime	Topbar **Studio** · studio rail · ⌘K “Open Runtime”	Active engine, endpoint, exec mode, probe of *that* backend
Workers	Studio rail · ⌘K “Open Workers” · Runtime card	Probe catalog; activate a backend (`endpoint` / engine)
Plugins	Studio rail · ⌘K “Open Plugins” · hidden topbar testid	Registry membership, Use, URL install, script library
Wire	Topbar **Studio** → Wire rail · ⌘K “Open Architecture”	Compose slots; Apply writes `activePlugins`
Settings	Topbar **Studio** → Settings rail · ⌘K “Open Settings”	Appearance, chart, live, MCP bridge, session keys, editor intel, theme

Active plugins live in `store.activePlugins`; per-plugin fields in `store.pluginsConfig` keyed by `` `${kind}:${id}` ``.

Conceptual model



Plugin Manager

Catalog

Studio Plugins catalog: sources, streams, engines, storage with Use

Lists sources, streams, engines, storages from the unified registry.

Action	Effect
Use	`setActivePlugin(kind, id)` for that row's kind
Capability badges	`offline`, `needsAuth`, `needsNetwork`, `needsProxy` from `plugin.capabilities`
Built-in flag	Built-ins resist casual unregister

Status bar shows active engine id and storage backend after changes.

Install (URL)

Studio Plugins Install: load an ES module URL or a same-origin example

Paste a module URL exporting a default plugin object (`kind`, `id`, `name`, contract methods). AXIS:

1. Fetches and validates export
2. Registers into the TypeScript registry
3. Persists install record so `restoreInstalledPlugins()` reloads next visit

Security notes (operator-level): only https/http schemes suitable for module load; treat third-party URLs as code execution in your browser origin. See security tests under `tests/security/`.

Supported URL kinds: **source**, **stream**, **engine**, **dataset**, **component**. Example plugins ship in `public/plugins/`:

- `example-coingecko-source.js`
- `example-tiny-pyne-engine.js`
- `example-cf-do-stream.js`

PYNE Agent (sister project — natural language → PYNE scripts) installs as a **component** from your deployed worker URL (or via Workers Manager → Install):

```
https://<pyne-agent-worker>/plugin/axis-pine-agent.js
```

See PYNE Agent and plugin reference. Works without the HOOX mesh.

Script Library tab

See Script library. The Plugins page hosts the panel so library and plugins share one studio surface.

Runtime vs Workers vs Plugins

One overlay (`AppPage` in `src/ui/studio/`) with a left rail. Switching rail pages does not close the overlay.

- **Runtime** — `src/ui/runtime/RuntimePage.tsx` — active engine, endpoint, exec mode.
- **Workers** — `src/ui/workers/WorkersPage.tsx` — backend inventory and probes.
- **Plugins** — `src/ui/plugins/PluginsPage.tsx` — catalog / install / library.

Workers

Studio Workers: backend inventory with pyne Pro API selected

Opens from the studio rail, ⌘K **Open Workers**, or a Runtime related card.

Sub-tab	Role
Overview	Health cards (distinct icon per worker) for pyne Pro API, AXIS Worker (prod + local wrangler), Pyodide, PWA service worker, optional PYNE Agent / pyne-worker
Detail	**Usage** (when to pick this worker), probe latency, feature flags, Use-as-backend / preload / install actions
Install	**When to use** + step-by-step setup with copyable commands (`make run` , `wrangler` , plugin URL, …)
Configure	Paste Backend URL, presets (`:5002` , `:8787` , workers.dev, axis.hoox.sh), activate Pyodide, install agent plugin

Catalog items (usage)

Worker	Icon intent	When to use
pyne Pro API	server	Primary **Server** calculation backend (Flask `:5002`). Best for compile/Numba and long history.
AXIS Worker	zap	Production edge data plane (on-chain proxy, scripts D1, optional `/api/run`). Default for On-Chain.
AXIS Worker (local)	activity	Local wrangler `:8787` while developing the Worker.
Pyodide	cpu	In-browser offline calc — no Backend URL; first load ~20–30s.
Service Worker	wifi	PWA shell cache (auto in production; skipped in Vite dev). Not a calc engine.
PYNE Agent	download	Optional NL → Pine plugin; scripts still run via your engine.
pyne-worker	settings	Optional HOOX mesh edge evaluator; paste origin as Backend URL.

Probes

- Run once when the modal opens; **Refresh** re-runs.
- Each worker has a **hard wall-clock timeout** (~5s) so a hung host cannot leave “Probing…” forever.
- Pyodide probe uses **HEAD** (not a full `pyodide.js` download).
- HTTP probes hit `GET /health` (JSON markers). Implementation: `src/workers/*` , `src/ui/workers/WorkersPage.tsx` . Operator CLI twin: AXIS CLI.

Hardened VPS note: production UFW typically allows only **22 + 443**. Pro API binds `127.0.0.1:5002` — public `:5002` probes correctly **time out**. On `https://axis.hoox.sh` , Workers Manager probes pyne Pro via **same-origin** `https://axis.hoox.sh/health` (nginx reverse-proxy), not loopback. See VPS demo.

Runtime (engine)

Engine, execution mode, Backend URL, Test, and Prefer WebSocket live on **Runtime**, not Settings. Storage is a **Wire** slot.

Field	Behavior
Engine	Registry engine list; labels via `engineOptionLabel`
Execution mode	When the active engine exposes `configSchema.mode` (server): `interpret` | `compile` | `auto`. Stored in `pluginsConfig.engine:<id>.mode` and sent on each run (WS and REST).
Prefer WebSocket run	Server engine: prefer `/ws/run` over REST when available
Backend URL	Shown when engine is `server` or has `configSchema.endpoint`
Test / Probe	`GET` health against endpoint; status message

Settings

Studio Settings General: appearance, chart labels, live stream, results

Product chrome only. Engine/endpoint/mode are on Runtime.

Field	Behavior
Default interval	Updates store; may reload bars for current symbol
Watchlist refresh	Clamp 5–120 seconds
Chart theme	Preset picker (void / classic / mono / boutique dark & light) — see `src/theme/presets.ts`
History bars	One-shot Load depth (venue may clamp further)
MCP	**Connect this tab to MCP** (default on). Requires a Worker API key under Settings → General → Worker (cloud + MCP). Status line shows `open` / `idle` / `error`. See MCP (agents).
Editor (Studio → Settings → Editor)	Pre-eval idle delay, block Run on errors, **Warn if //@version is missing** (`W001` / `W002`), **Warn on v6-removed kwargs** (`W104` : `when=` / `transp=`). Offline / Pyodide match Pro API diagnostics. Symbol navigation: `F12` / `Shift-F12` / `F2` .

Deep multi-page history and gap repair live in the Data Source Manager, not Settings.

Execution mode maps to PYNE `Runtime.run(..., mode=…)` :

Mode	Behavior
`interpret`	Full AST interpreter
`compile`	Numba/numpy compiled path (first run JITs; Flask is not prewarmed)
`auto`	Default on Server / Pyodide: try compile, fall back to interpret on failure

The Connection HUD engine chip shows the selected mode. Save persists via Solid store `persist()` into `pynescript.axis.v1` . Escape closes; Ctrl/Cmd+Enter saves.

Pyodide engines hide endpoint—calculation is same-origin assets, not Flask.

Interface surface (UI map)

Control	Store field
Engine select	`activePlugins.engine`, flat `engine`
Storage select	`activePlugins.storage`
Endpoint	`endpoint`
Source/stream topbar	`activePlugins.*`, `source`, `live.streamId`
Theme toggle	`theme` + `data-theme` on `<html>`

Internals

Path	Role
`src/ui/studio/`	Full-page overlay kit (`ax-*`)
`src/ui/runtime/RuntimePage.tsx`	Active engine / endpoint / mode
`src/ui/plugins/PluginsPage.tsx`	Catalog / install / library
`src/ui/workers/WorkersPage.tsx`	Backend inventory / probes
`src/ui/settings/SettingsPage.tsx`	Product chrome
`src/ui/wire/WirePage.tsx`	Compose recipes
`src/ui/plugin-badges.tsx`	Capability badges
`src/plugins/registry.ts`	Unified registry
`src/plugins/loader.ts`	URL load + restore (incl. component)
`src/plugins/bootstrap.ts`	Built-ins

Invariants

1. Settings never write script bodies—only layout/config.
2. Catalog **Use** does not auto-Run; Load/Run remain explicit.
3. Source change may re-default stream (`defaultStreamForSource`).
4. API keys for cloud/git belong in `pluginsConfig`, not in git-committed defaults.

Worked examples

Point AXIS at local Worker

1. Settings → Engine Server-Side → Endpoint `http://127.0.0.1:8787`
2. Probe → OK
3. Save → Run

Install example source

1. Manager → Install → URL to `.../plugins/example-coingecko-source.js` (served origin)
2. Catalog → Source appears → Use → Load

Failure modes

Symptom	Fix
Empty catalog	Built-ins not registered—hard reload; check console
URL install fails	CORS on plugin host; bad export; mixed content
Probe fails	Backend down; wrong path; HTTPS mixed content
Settings not sticky	localStorage blocked / quota

See also

- Script library
- Compose recipes
- Plugins

PYNE AGENT

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "PYNE Agent" description: "Write scripts from natural language (PYNE Agent) in AXIS using the optional pyne-agent-worker Cloudflare® plugin."

PYNE AGENT

What it does

Describe an indicator or strategy in plain language. The **PYNE Agent** (Cloudflare® Workers AI™) returns a full script you can insert into the AXIS editor, then **run** with your normal engine (local PYNE Pro API, edge worker, or Pyodide).

You do **not** need the full HOOX mesh. The agent worker is usable **standalone**.

Pine Script™ and TradingView® are trademarks of TradingView®, Inc. Cloudflare® is a registered trademark of Cloudflare®, Inc.

Prerequisites

1. A deployed pyne-agent-worker origin (or local `wrangler dev`).
2. AXIS open in the browser (dev or production PWA).

Optional: API key if the worker has `API_KEY` set.

Install the plugin

1. Open **Manager** → **Plugins** → **Install**.
2. Paste the **same-origin** module (avoids cross-origin module CORS):

```
/plugins/axis-pine-agent.js
```

Or the remote copy (needs CORS on the agent Worker):

```
https://pyne-agent-worker.cryptolinx.workers.dev/plugin/axis-pine-agent.js
```

3. Set **endpoint** to the agent Worker origin (no trailing slash), e.g. `https://pyne-agent-worker.cryptolinx.workers.dev`. Same-origin install still talks to this API host. Workers Manager → PYNE Agent also has an editable **Agent endpoint** (URL + API key).
4. Set **API key** if required.
5. Prefer **v6** / **strategy** or **indicator** when you know the target style.

6. In **Workers Manager → PYNE Agent**, pick a **Persona** (`auto` detects per message): `pine` writes scripts, `axis` answers anything about the app and can change it for you (theme, layout, runs), `trader` does market analysis and risk coaching.

AXIS migrates a previously saved remote plugin URL to `/plugins/axis-pine-agent.js` on restore.

AXIS **loads component plugins from URL** (Manager → Install, or Workers Manager → install agent). You can also open the worker's built-in chat UI:

```
https://<your-agent-worker>/
```

or use the floating **PYNE Agent** button the plugin may inject after install.

Typical workflow

1. Ask for a script (example: "v6 RSI strategy with ATR trailing stop and clear plots").
2. Review the fenced Pine block.
3. **Insert into editor** (plugin action) or copy/paste.
4. Select your **engine** and **Run** as usual.
5. Iterate with follow-up messages ("use length input", "overlay false", ...).

Tips

- Prefer specific version (`v5` / `v6`) and script kind in the request.
- The agent may use a private knowledge base (docs + open examples). Quality improves after the operator ingests docs — not required to start chatting.
- Validation against **pyne-worker** is optional on the agent side; AXIS evaluation is still your engine plugin.

Troubleshooting

Issue	Fix
Load error	Check URL HTTPS, CORS, and that the module is reachable
Unauthorized	Match plugin `apiKey` to worker secret
Weak answers	Ingest KB on the worker; still usable without it
Cannot insert	Use copy; ensure plugin `api.insertScript` is provided by host

See also

- Plugin deep dive
- Manager and settings
- Plugin examples

DATA SOURCE MANAGER

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Data Source Manager" description: "Background multi-page OHLCV backfill to a past date, dataset validation, and gap fills — without blocking the chart."

DATA SOURCE MANAGER

Deep history accumulation for a **symbol + source + timeframe**, down to a **past date**. Work runs fully in the **background**: chart, live streams, and the editor stay free.

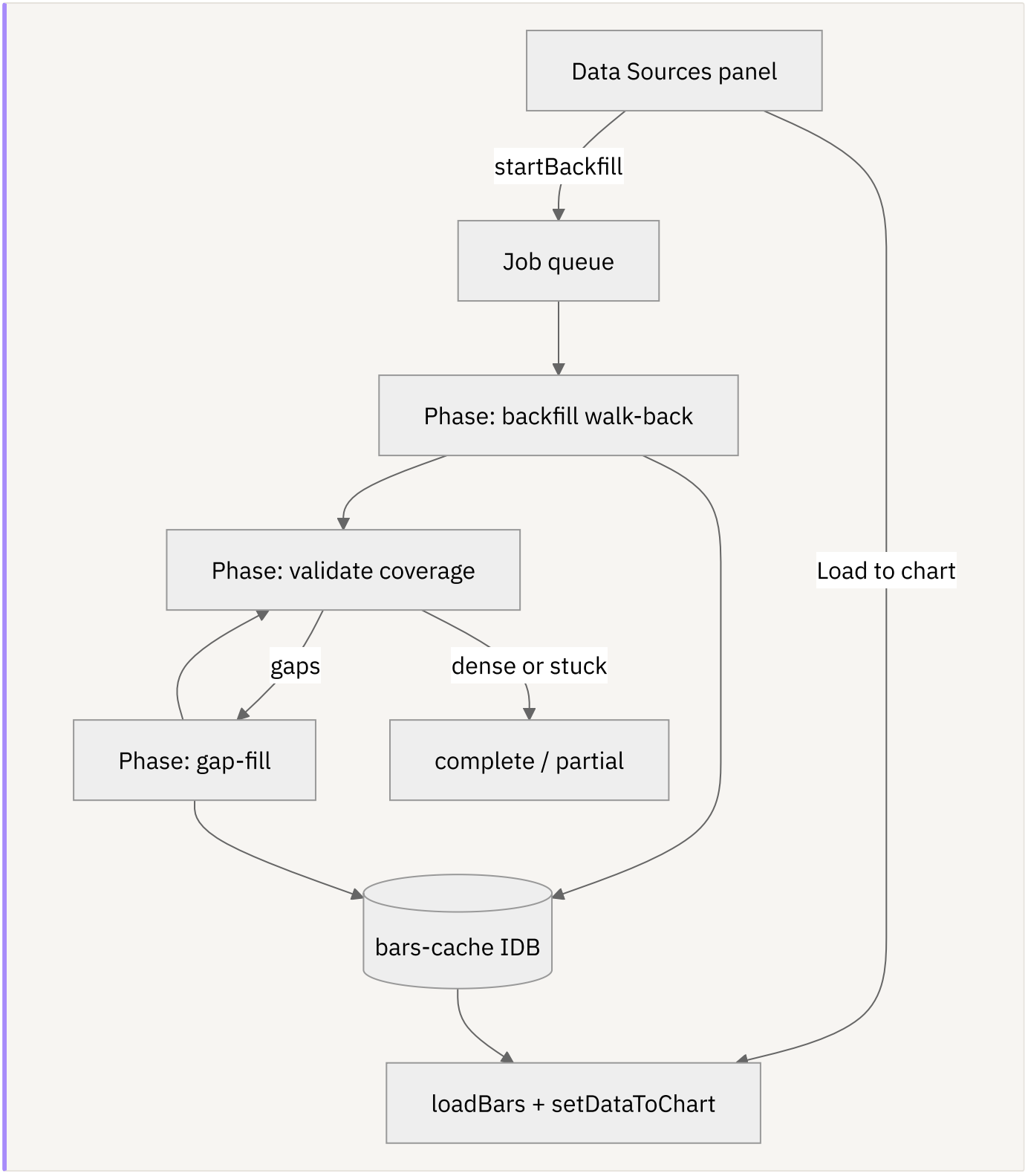
Abstract

Piece	Role
Topbar **Load**	One venue page (or `historyBars`) straight onto the chart
Data Sources panel	Queue backfill jobs, watch progress, load cache onto chart
Walk-back pages	Multi-page REST with `endTime` only (avoids start+end one-page traps)
Validate + gap-fill	After backfill, check density and re-download holes
IDB bars cache	Durable OHLCV per `source|symbol|interval`

Open from the topbar **Data** button, or command palette → **Toggle Data Source Manager**.

Data Sources panel: symbol, venue, timeframe, and background backfill

Conceptual model



How to use

1. Open **Data** (Data Sources panel).
2. Set **symbol**, **source / exchange**, **timeframe**, and **accumulate to** (UTC date). Symbol and timeframe follow the chart until you edit them.
3. Optionally check **Apply to chart when complete**.

4. Click **Start background backfill** — the job id appears immediately; work continues detached.
5. Watch phases: **Backfilling** → **Validating** → **Filling gaps** → **Complete** (or **Partial**). Filter jobs with All / Active / Done / Issues.
6. Click **Load to chart** when ready (or rely on apply-when-complete).
7. **Browse datasets** (header icon or the button) opens the Dataset manager: coverage map, date range / max bars, then load onto the chart.

Pause / resume / cancel apply between pages. Multiple jobs queue with low concurrency (rate-limit friendly).

Deleting datasets

Delete in the Dataset manager removes the dataset in full — bars *and* metadata — from every persistence sink (session / local / git / worker), so the row does not come back on the next refresh.

To wipe every dataset at once, use **Settings** → **Data** → **Dataset manager** → **Flush dataset manager**. The flush clears the memory mirror, the local IndexedDB cache, and the remote dataset stores (git / cloud). Bars already painted on the chart stay until the symbol is reloaded.

Phases

Phase	Behavior
Backfill	Walk newest → older using `endTime` + page limit only. Never send `startTime` + `endTime` together to venue APIs that return the *first* N bars from start (would falsely complete in one page).
Validate	Measure coverage from target past date → job end time: leading, internal, and trailing holes via interval step + tolerance.
Gap-fill	For each gap window, walk-back download again; re-validate (several rounds).
Done	`datasetComplete` when dense; otherwise **Partial** with remaining gap count (venue may lack older history).

Job fields (UI)

Field	Meaning
Bars / Pages	Cached bar count and network pages this job
Oldest / Target	Current oldest open time vs past-date target
Gaps	Detected holes · how many fill attempts landed bars
Coverage	full / partial / in progress

Relation to Topbar Load

	Topbar Load	Data Source Manager
Blocks UI	Yes (status loading)	No (background jobs)
Depth	Single page / `historyBars`	Multi-page to past date
Chart	Always paints	Only on **Load to chart** or apply-when-complete
Cache	Ephemeral `store.bars`	Durable IDB `bars-cache`

Use Load for desk-speed symbol switches; use the manager when you need years of 1m/1h/1d history for research.

Internals

Path	Role
`src/data/data-source-manager.ts`	Job queue, walk-back, validate, gap-fill
`src/data/bars-cache.ts`	Memory-first + IDB durable OHLCV cache
`src/data/dataset-store.ts`	Dataset mirror + delete / flush entry points
`src/data/dataset-sinks.ts`	Persistence sinks (session / local / git / worker)
`src/data/bars-gaps.ts`	Gap detection / coverage report
`src/ui/DataSourceManagerPanel.tsx`	Panel UI
`src/ui/CachedDatasetsModal.tsx`	Dataset manager modal
`src/ui/dsm/`	Job card, coverage map, shared formatters
`src/plugins/types.ts`	`SourceOpts`: `startTime?`, `endTime?`, `signal?`
`src/sources/catalog.ts`	Venue date params + `sourcePageLimit`

bars-cache performance (2.0.1+)

`bars-cache.ts` is **memory-first**:

API	Behavior
`getCachedBars`	Serve warm in-memory series immediately; cold miss hydrates from IndexedDB then fills memory
`getCachedBarCount`	Count without cloning the full series when warm
Range / window	`sliceBarsForLoad` + count helpers avoid full clones for progress UI
Soft caps	`BARS_CACHE_MAX` (100k bars/series), `BARS_CACHE_MAX_SERIES` (48 keys)

DSM gap-fill progress uses `getCachedBarCount` instead of loading full series each tick.

Limits & safety

- Max concurrent network jobs: **1** (queue the rest).
- Caps: pages per walk, bars per series (soft IDB cap), gap-fill rounds.
- Backfill forces `fallback: false` so synthetic walks do not fake venue history.
- Cancel uses `AbortController` on in-flight fetches (symbol / job switches abort mid-page).

See also

- Sources — built-in historical plugins
- Quick start — one-shot Load
- UI shell — topbar panels

ON CHAIN

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "On-Chain data" description: "DefiLlama protocol TVL, GeckoTerminal DEX pool OHLCV, and derived TVL spike events — not a wallet, not CEX-only price." sidebarTitle: "On-Chain data"

ON-CHAIN DATA

AXIS **on-chain data plane**: search public protocol and DEX metrics and plot them next to your chart. Built-in providers include **DefiLlama protocol TVL** (total value locked, USD), **GeckoTerminal DEX pool OHLCV**, and **derived TVL spike/drop events**.

On-Chain panel with Aave TVL attached beside the chart

Abstract

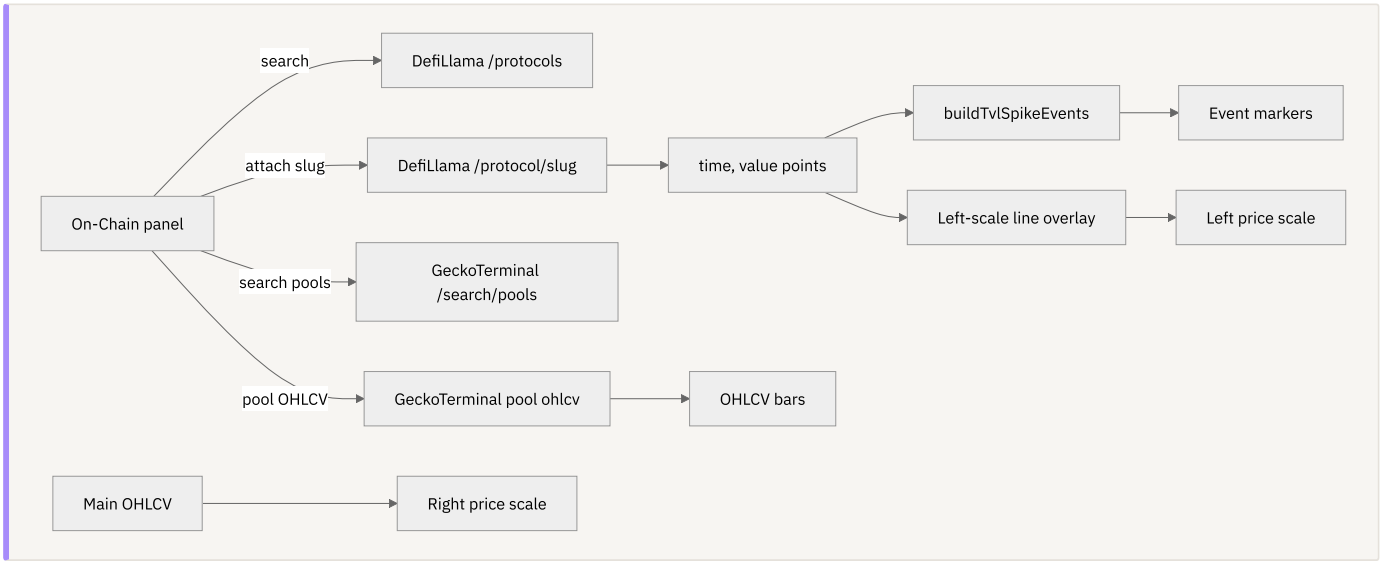
Piece	Role
On-Chain panel	Search protocols / pools, attach / hide / detach series
DefiLlama TVL	Daily USD liquidity history for a protocol slug
GeckoTerminal OHLCV	DEX pool candlesticks (minute / hour / day aggregates)
TVL spike events	Day-over-day |%| threshold events from attached TVL (`tvl_spike` / `tvl_drop`)
Left price scale	Independent metric axis so main OHLCV keeps the right scale
Provenance	Each series records provider + finality / snapshot notes

Open from the topbar **Panels** group (**On-Chain**), or command palette → toggle the On-Chain panel (panel id `onchain`).

What this is not

- **Not a wallet.** AXIS does not connect to MetaMask, Ledger, or any signing wallet for this plane.
- **Not CEX price by default.** Protocol TVL is liquidity in USD, not the exchange candle series for the chart symbol. DEX pool OHLCV is **on-chain pool** price, which can diverge from centralized venues.
- **Not Pine.** These overlays are host-side dataset series; they are not produced by `indicator()` / `strategy()` plots.

Conceptual model



How to use (DefiLlama TVL)

On-Chain panel: TVL / DEX / Events tabs and popular protocol presets
Attaching Aave TVL from the On-Chain panel onto the chart

1. Load a market as usual (symbol + source + timeframe).
2. Open **On-Chain**.
3. Search a protocol (e.g. `aave` , `uniswap` , `lido`).

4. Click a result to **attach** its TVL history.
5. The curve appears on the **left scale** of the price pane (when overlay wiring is active).
6. Hide or remove series from the **Attached series** list; **Clear** removes all.

Multiple protocols can be attached (chart safety caps the number of on-chain lines).

Refresh, export, popular presets

Command palette (**⌘K** / **Ctrl+K**) exposes host jobs for attached TVL series — no Pine required:

Command id	Title	What it does
`onchain.refresh`	On-Chain: Refresh Attached TVL	Re-fetch every attached DefiLlama TVL series (`refreshAllAttachedTvl`)
`onchain.export.series`	On-Chain: Export Series CSV	Download **all** attached series as long CSV (`series,time,value`)
`onchain.attach.popular`	On-Chain: Attach Popular TVL	Attach the top **5** protocols by current TVL (`attachPopularTvl(5)`)

Related palette actions (panel / hygiene):

Command id	Role
`panel.onchain`	Toggle On-Chain panel
`onchain.mode.tvl`	Open panel (TVL mode entry)
`onchain.clear.series`	Detach all series
`onchain.clear.events`	Clear event markers plane
`onchain.health`	Probe Worker `/api/onchain/health` (Connection HUD)

Export format: one row per point — `series,time,value` — with `time` as unix seconds and `value` as USD TVL (or other scalar). Empty attachments still download a header-only file.

Popular preset: empty DefiLlama protocol browse (highest TVL first), capped by the chart series limit (`MAX_ONCHAIN_SERIES` , currently 8). Already-attached slugs refresh in place rather than duplicating.

Refresh: re-runs the DefiLlama protocol TVL fetch for each attachment so left-scale lines stay current after long sessions.

DEX pools (GeckoTerminal)

On-Chain DEX tab: GeckoTerminal pool OHLCV next to an attached TVL overlay

Phase 2 adds **DEX pool OHLCV** via GeckoTerminal public API (no API key). Use this when you want on-chain pool candles rather than protocol TVL:

Concept	Detail
Network	Gecko network id (`eth` , `solana` , `base` , `arbitrum` , `polygon_pos` , …)
Pool address	EVM `0x` + 40 hex, or Solana-style base58
Timeframe	`minute` / `hour` / `day` with aggregate (e.g. 1h → hour + aggregate 1)
Currency	Default USD when proxied with query pass-through
Page size	**Max 1000 candles per request** (`limit`); Gecko hard cap
Walk-back	`endTime` → Gecko `before_timestamp` (bars **strictly before** that unix second)

Client helpers live in `src/onchain/geckoterminal.ts` (`fetchGeckoPoolOhlcv` , `searchGeckoPools` , interval/network maps). Browsers should use the **Worker gecko proxy** (below) because public GeckoTerminal hosts often lack CORS for arbitrary origins.

Deep history (Data Sources + GeckoTerminal DEX)

Topbar **Load** fetches a **single page** (up to 1000 bars). For multi-page history down to a past date:

1. Set the chart source to **GeckoTerminal DEX** (`geckoterminal-ohlcv`).
2. Use a pool symbol such as `eth:0x…` (network + pool address).
3. Open **Data** (Data Source Manager) → set symbol / source / timeframe / **accumulate to**.
4. Start background backfill — AXIS walks older pages with `endTime` only (each page maps to `before_timestamp`). The source does **not** multi-page inside one `fetchHistorical` call; DSM does it page-by-page.

See Data Source Manager for job phases and cache behavior.

Events (TVL spikes)

From an attached TVL scalar series, AXIS can derive **day-over-day** percent-change events:

Field	Behavior
Type	`tvl_spike` (gain) or `tvl_drop` (loss)
Default threshold	10% absolute day-over-day change
Severity	`warn` below critical band; `critical` for large moves (default ~25% or 2.5× threshold)
Payload	`pctChange` , `prevValue` , `value` , thresholds

Pure helper: `buildTvlSpikeEvents` in `src/onchain/events.ts` . These are **derived** from daily snapshots — not mempool or block-final chain events.

DefiLlama **raises** and **token unlocks** require Pro API (`pro-api.llama.fi`) and are **not** fetched by the free Worker proxy; inject external events only if you already have that data.

Scale and chart behavior

Behavior	Detail
Scale	On-chain TVL lines use the built-in **left** price scale (`left`)
Main market	Primary OHLCV stays on the **right** scale
Compare	Compare overlays may share the left scale; on-chain keys are `onchain_*` , not `overlay_*`
Units	TVL values are **USD** liquidity totals from DefiLlama; DEX OHLCV is pool quote (often USD)

Settings

Settings → **General** → **On-Chain** is a short note only (no extra bindings):

- By default the PWA calls **DefiLlama / GeckoTerminal directly** (public CORS).
- Worker proxy (`{endpoint}/api/onchain/llama` / `…/gecko`) is used **only** when Backend URL is a real AXIS Worker (`worker.axis.hoox.sh` , `worker-axis` / `*.workers.dev` , or wrangler `:8787`).
- **Not a wallet** — public metrics only; no signing

Keep **Backend URL** as the Pro API for Pine (`https://axis.hoox.sh` or local `:5002`). That host is **not** an on-chain proxy.

Network path (CORS / Worker proxy)

Public llama.fi / geckoterminal APIs currently allow browser origins (`Access-Control-Allow-Origin: *`), so AXIS **defaults to direct fetch**.

Using `https://axis.hoox.sh/api/onchain/…` is wrong: nginx serves the **SPA HTML**, which produces *invalid JSON / got HTML*. The client now avoids that host for on-chain bases.

When Backend URL **is** an AXIS Worker, paths map as:

Client request	Worker route	Upstream
`{endpoint}/api/onchain/llama/protocols`	allowlisted GET	`https://api.llama.fi/protocols`
`{endpoint}/api/onchain/llama/protocol/{slug}`	allowlisted GET	`https://api.llama.fi/protocol/{slug}`
`{endpoint}/api/onchain/gecko/networks/{net}/pools/{addr}/ohlcv/{tf}`	allowlisted GET (+ query)	GeckoTerminal OHLCV
`{endpoint}/api/onchain/gecko/search/pools`	allowlisted GET (+ query)	GeckoTerminal search
`{endpoint}/api/onchain/health`	local	feature flags

- Local Worker: `http://127.0.0.1:8787` + `cd worker && bun run dev` if you want the allowlisted proxy.
- Override with plugin `config.baseUrl` when needed.

Isolate memory caches short-TTL responses (`X-Axis-Onchain-Cache: HIT|MISS`):

Route class	TTL (approx.)
Llama protocols list	10 min
Llama protocol detail	2 min
Gecko OHLCV	60 s
Gecko search	120 s

Provenance and finality

Each attached series carries a short provenance line, for example:

- Provider: **DefiLlama · protocol TVL (USD)**
- Provider: **GeckoTerminal · pool OHLCV**
- Finality: **Daily snapshot · not CEX price** (TVL) or pool candle finality notes (DEX)

Treat DefiLlama points as **daily** aggregates from the public API — not block-final on-chain events and not exchange marks. DEX candles reflect the **pool** at GeckoTerminal's sampling, not a CEX order book.

Network, CORS, and proxies

Situation	What to do
invalid JSON / got HTML for aave	Backend URL is a SPA host (e.g. `axis.hoox.sh`) — leave it for Pine; on-chain should hit llama.fi directly (reload after upgrade)
Local dev blocked by CORS	Point Backend URL at wrangler `:8787` Worker, or set plugin `baseUrl`
Self-hosted demo	Optional: reverse-proxy allowlisted `/api/onchain/*` to the Worker
Offline	Needs network; there is no offline synthetic TVL/DEX walk by default

See also CORS and origins for Worker origin allowlisting.

Dataset plugins (advanced)

On-chain providers implement the structural **dataset** plugin contract (`fetchDataset`). Built-ins register via the on-chain catalog; dynamic URL plugins may install additional datasets. End users normally only use the On-Chain panel — see Datasets for the contract sketch.

Related docs

- Data Source Manager — OHLCV backfill (separate from on-chain scalars)
- Sources — historical bar sources
- Worker overview — `/api/onchain/…` route table
- CORS and origins — browser cross-origin constraints

SCRIPT LIBRARY

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Script library" description: "Save, load, import/export Pine scripts via local IndexedDB, cloud Worker, or git (GitHub/GitLab)."

SCRIPT LIBRARY

The script library is the operator-facing surface of **storage plugins**. The editor holds the working document; the library is durable, named documents with metadata.

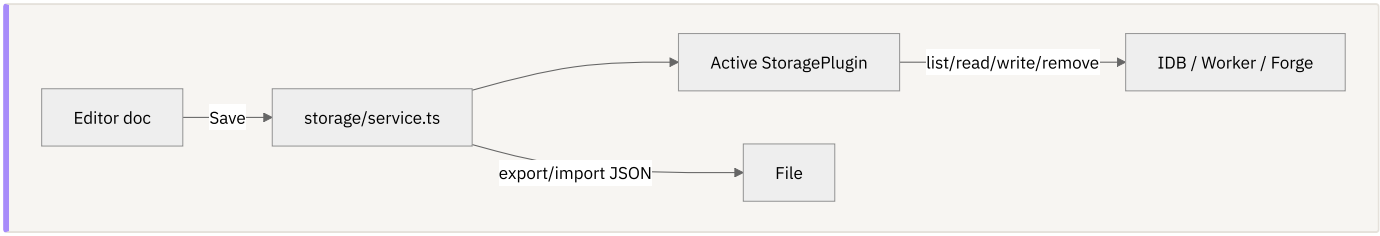
Script Library panel with local IndexedDB storage

Abstract

Backend	Id	Medium	Auth
Local	`local`	IndexedDB (`pynescript.axis.storage`), LS fallback	none
Cloud	`cloud`	Worker `/api/scripts` (D1 or memory)	Bearer Pro API key
Git	`git`	GitHub/GitLab Contents API	PAT

Active backend: `store.activePlugins.storage` (Settings or library picker).

Conceptual model



Service layer (`listScripts` , `readScript` , `writeScript` , ...) never talks to engines.

Interface surface

Manager → **Script Library**:

Control	Behavior
Backend select	Prompts to **copy** scripts onto the new engine (source is never deleted)
Refresh	`list()`
Name / description	Metadata for next write
Save	Write current editor doc
Load	`read` → inject editor (`loadLibraryDoc` / setDoc)
Delete	`remove`
Export JSON	Full library dump
Import JSON	Bulk write (`importLibraryJson`)
Cloud endpoint + key	Settings → Script storage **and** this panel. Saved under `pluginsConfig['storage:cloud']`
Version history	Every engine (local snapshots, Worker revisions, git commits). Open / Restore on a card
Git fields	provider, token, owner, repo, branch, basePath, ...

Status line: backend · remote · branch · connected · count.

Built-in studies (always in the app)

Original AXIS Pine catalog — apply from **Scripts** → **Built-in** or ⌘K **Add built-in script...**. Not TradingView® templates. See Built-in scripts.

Pine Script™ v6 starter pack

Repo ships a portable library dump of **v6** language examples (libraries, enums, MTF):

Path	Role
`examples/script-library-starter.json`	Importable library JSON (`//@version=6` helpers + sample studies)

Import: Manager → Script Library → **Import JSON** → pick the file (or any prior export). Uses `importLibraryJson` with new ids so it merges into the active backend (local / cloud / git).

Prefer `//@version=6` for new scripts; engines accept v5/v6 when the runtime supports them.

Local backend

- DB name `pynescript.axis.storage` v1 stores `scripts` + `kv`
- Older library keys migrate once
- Drafts: optional `saveDraft` / `loadDraft` without polluting remote backends
- Works fully offline

Invariant: browser profile wipe deletes local library—export JSON for backup.

Cloud backend

- `Authorization: Bearer <api_key>`
- Scripts partitioned by key hash on Worker
- Concurrent updates may use `If-Match` / revision → HTTP 409 on conflict
- Default endpoint falls back to `store.endpoint` or `http://127.0.0.1:8787`

Configure key in library panel or Settings-related config; keys stay in this browser's localStorage state blob.

Git backend

Invariant	Detail
Save commits	Each write/remove is a commit (+ push via API)
Drafts local	No remote commit for draft buffer
basePath	Default `pine-library/`
Providers	`github`, `gitlab` (+ self-hosted `apiBaseUrl`)

Token scopes: GitHub `contents:write`; GitLab `api` or `write_repository`.

Published libraries (import emulator)

`library("Name")` scripts can be **published** as versioned folders so other scripts can `import owner/Name/1 as alias`:

```
{basePath}/published/index.json
{basePath}/published/{namespace}/{Name}/{version}/lib.pyne
{basePath}/published/{namespace}/{Name}/{version}/manifest.json
```

- Library panel → **Publish library** (or a successful **Run** of a `library()` — auto, skipped if unchanged).
- Namespace defaults to the git owner, else `user`.
- Local cache always updates so Pyodide / offline runs resolve imports.
- The engine receives `{namespace, name, version, source}` and pyne registers them before `import` (interpret path).

Worked examples

Solo offline library

1. Storage = local

2. Write indicator in editor
3. Name `sma-ribbon` → Save
4. Reload page → Library → Load

Team cloud

1. Admin creates key on Worker
2. Storage = cloud; paste endpoint + key
3. Save shared strategies
4. Second browser: same key → list

Research monorepo

1. Storage = git; GitHub owner/repo; basePath `research/pine`
2. Save → commit message from template
3. PR review on GitHub UI

Internals

Path	Role
`src/ui/ScriptLibraryPanel.tsx`	UI
`src/storage/service.ts`	Facade
`src/storage/local.ts`	IDB
`src/storage/cloud.ts`	Worker API
`src/storage/git.ts`	Git plugin
`src/storage/git-github.ts`, `git-gitlab.ts`	Forges
`examples/script-library-starter.json`	Pine v6 starter pack

Contract: Plugin types `StoragePlugin`.

Failure modes

Symptom	Cause	Mitigation
Empty list after switch	Wrong backend / auth	Check status line + key
401 cloud	Bad/missing key	Regenerate; re-save config
409 cloud	Revision conflict	Re-read then write
Git 404	Wrong owner/repo/basePath	Fix config
Quota local	Huge scripts in LS fallback	Prefer IDB; export prune

See also

- Manager and settings
- Compose recipes — git / edge

- State namespaces

BUILTIN SCRIPTS

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Built-in scripts" description: "Original AXIS studies you can apply from the Scripts panel — not third-party chart-host templates."

BUILT-IN SCRIPTS

AXIS ships a **first-party** catalog of Pine `indicator()` / `strategy()` scripts so you can apply common studies without pasting or importing anyone else's source.

Every built-in starts with:

```
//@version=6
// AXIS first-party built-in. Original source — not a TradingView® template.
```

They wrap public technical-analysis formulas through PYNE `ta.*` (SMA, RSI, ATR, …) or short original implementations of well-known public recurrences (McGinley Dynamic, Fisher Transform, floor pivots). **AXIS Drawings** is the overlay example for `line`, `linefill`, `box`, `label`, `polyline`, and `table`. They are **not** copies of TradingView® built-in templates.

Apply

Where	How
Scripts panel	**Built-in** → search → click a row. Runs onto the chart as a new instance.
Add ▾	Topbar or editor Add chevron → **Add built-in script…**
⌘/Ctrl+K	**Add built-in script…**, or type `rsi` / `macd` / `vwap` (two+ characters) to jump.

To inspect a built-in already on the chart, click the **file** icon on its pane badge (second, after settings) — that loads the source into the editor.

Applied scripts persist with the workspace like any other Run. Open Script Settings on the card to change inputs.

Inventory

Grouped by catalog category. `covers` is the public TA idea, not a vendor product name.

Moving averages

SMA, EMA, WMA, RMA (Wilder), VWMA, Hull, DEMA, TEMA, ALMA, KAMA, LSMA, median, SWMA, McGinley Dynamic, MA ribbon, percent envelope, MA cross (study).

Channels

Bollinger Bands, %B, bandwidth, Keltner, Donchian, linear-regression channel / curve, short-vs-long Bollinger width.

Oscillators

RSI, Stochastic, Stoch RSI, MACD, CCI, momentum, ROC, Williams %R, Awesome Oscillator, Ultimate Oscillator, TRIX, TSI, CMO, Fisher Transform, Coppock, KST, Relative Vigor, Relative Volatility, DPO, PPO, PVO, Balance of Power, Bull/Bear Power, Connors RSI, SMI, Woodie CCI, RCI, Ulcer Index, Mass Index, Choppiness, trend strength (ADX).

Trend

Supertrend, ADX, DMI, Aroon / Aroon Osc, Parabolic SAR, Vortex, Ichimoku, Chande Kroll stop, Chandelier exit, volatility stop, Alligator, fractals, ZigZag.

Volume

Volume, OBV, VWAP, VWAP bands, TWAP, CMF, Chaikin Oscillator, MFI, A/D, Ease of Movement, Elder Force, Klinger-style signed-volume EMAs, NVI, PVI, net volume, PVT.

Volatility / pivots / session

ATR, true range, historical volatility, average range; floor pivots from the prior day, pivot high/low; close-vs-volume correlation; session background.

Strategies

SMA cross, MACD cross, RSI mean-reversion, Bollinger bounce, Supertrend follow, SAR flip, Keltner break, Donchian break, momentum, stochastic cross, bar up/down, consecutive closes, inside bar, outside bar, pivot reversal, ATR expansion.

What AXIS does not ship

Some names that appear on closed chart hosts are **skipped** on purpose (proprietary scoring, extra market data, tick aggressor feeds, or chrome AXIS already has):

Skip	Why
Technical Ratings	Proprietary composite
Dividend yield / open interest	Extra feeds
Compare / multi-time-period charts	Host layout (AXIS Compare + multi-chart)
Volume delta / CVD / up-down volume	Tick / aggressor data
Auto Fib / pitchfork	Drawing tools
Moon phases / seasonality / price target	Calendar or analyst data
Advance/decline breadth	Market-wide series
Chop Zone	Vendor-specific coloring

See `src/indicators/builtins/skipped.ts` for the full skip list.

Library starter pack vs built-ins

Pack	Role
Built-ins (`src/indicators/builtins/`)	Always in the app; apply from Scripts / ⌘K
`examples/script-library-starter.json`	Importable v6 **language** examples (libraries, enums, MTF) for the Script Library

Prefer built-ins for “add RSI to this chart”. Prefer the starter pack when learning Pine v6 library/enum patterns.

See also

- Scripts (indicators & strategies)
- Script library
- Quick start

STRATEGY AND RESULTS

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Strategy and results" description: "How AXIS turns engine events into trades, stats, chart markers, equity, and exportable Results tabs."

STRATEGY AND RESULTS

After **Run**, the engine returns a structured payload. AXIS **AXIS** normalizes events, pairs trades, paints markers, and exposes a Results drawer. Strategy math here is a **viewer**—not a brokerage.

Results overlay showing strategy stats, equity curve, and closed trades

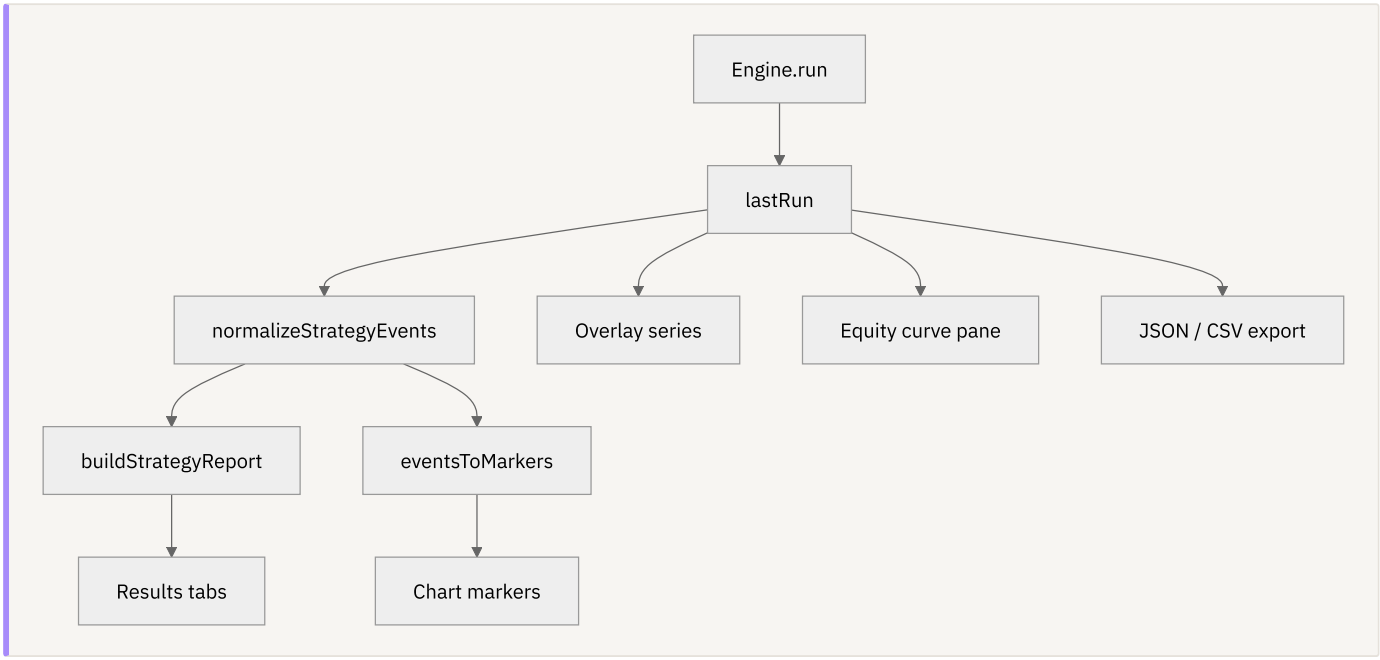
Abstract

Engine `RunResult` (simplified):

```
status: success | error
plots[] | series{}
events[]     — entries, exits, orders, custom
drawings[]   — Pine line/label/box objects
meta{}       — ms, overlay, script_name, plot_meta, …
error?
```

AXIS stores this as `store.lastRun` (in-memory; not fully rehydrated from localStorage).

Conceptual model



Results drawer tabs

Strategy tab: stats cards, equity curve, and closed-trades table

Tab	Operator value
Events	Chronological normalized stream (kind, time, price, id, dir) with a **Stream / Open⇄Close view switcher** — the Open⇄Close view groups events into pyramiding-aware position cycles (avg-entry parity with the pyne broker, per-fill P&L, jump-to-bar)
Strategy	Single column: stats → equity → closed trades below, with return %, bars-held, and fill-count detail
Optimise	Strategy-only `input.*` search — Hyperparameter Optimisation
Plots	Series inventory (points, last value)
Metrics	Runtime, status, engine, source, bars, high-level stats
Raw	Pretty-printed JSON for support / diffs

Saved runs persist a JSON-safe strategy stats snapshot, so script-library rows show win %, profit factor, return %, max drawdown, and average trade at a glance. A shared walker (`walkStrategyEvents` in `src/results/strategy.ts` , positions in `src/results/positions.ts`) feeds trades, stats, positions, and the events stream.

Strategy metrics

Computed by `buildStrategyReport`:

Stat	Definition (viewer)
totalPnl	Sum of closed trade PnL (price units)
winRate	% wins among closed trades
profitFactor	Gross profit / gross loss
avgTrade / avgWin / avgLoss	Means over closed set
maxDD	Peak-to-trough on cumulative trade PnL path
wins / losses / trades	Counts

Pairing rules: entry-like kinds open by `id`; exit/close kinds match id or sole open trade. Direction flips short PnL. Missing prices drop the event from pairing (with normalization attempting OHLC fill from bars when available).

Chart coupling

- Trade markers on the price pane (`eventsToMarkers`) show filled size as `Long N` / `Short N` (exits: order id + qty); missing engine qty keeps the id / `L` / `S` / `X` fallback
- Scroll-to-time on trade row click
- Equity curve series when strategy events warrant (`buildEquityCurve`)
- Status bar snippet: closed trade count · net PnL when available

Export

Export	Content
JSON	Full `lastRun`
CSV	Closed trades (`tradesToCsv`)
Clipboard	CSV of trades when supported

Filenames: `axis-run-<ts>.json`, `axis-trades-<ts>.csv`.

Interface surface

- Drawer height persisted (`resultsPanel.height`); open state toggled after runs (`openResults` default true for interactive runs)
- Silent live re-runs can skip auto-open
- Equity / indicator panes created on demand when non-overlay or strategy paths need them

Worked example

1. Load `BTCUSDT` daily history.
2. Run a crossover strategy (see Quick start).
3. Results → Strategy: inspect win rate.
4. Click a green trade → chart jumps to entry.

5. Export CSV for a notebook.

Internals

Path	Role
`src/ui/ResultsPanel.tsx`	Drawer UI
`src/results/strategy.ts`	Trade pairing + stats
`src/results/events.ts`	Normalize events, markers, equity
`src/indicators/runner.ts`	`runAndApply` orchestration
`src/chart/series-factory.ts`	Plot colors / series

Language-level strategy semantics: PYNE runtime.

Invariants

1. Results are a function of **last successful/error payload + current bars**—changing bars without re-run can desync prices used for fill.
2. Viewer PnL is not currency-normalized; treat as relative tape units.
3. `meta.overlay === false` routes plots to an indicator pane, not price.
4. Pine drawings (script) are distinct from user drawing tools (Drawings).

Failure modes

Symptom	Explanation
Events but 0 trades	Only entries, or unpaired ids
Empty plots	Series all null / name filtered (`__` prefix hidden)
Markers missing	Manager not mounted or events lack times
Huge Raw JSON	Many events—export file rather than copy

See also

- Quick start
- AXIS — results and strategy
- AXIS — indicators

HYPERPARAMETER OPTIMISATION

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Hyperparameter Optimisation" description: "Search strategy input.* values over N trials (TPE / random / grid) with holdout or walk-forward. Strategies only."

HYPERPARAMETER OPTIMISATION

AXIS can search Pine Script™ `input.*` values on a **`strategy()`** script and apply the best assignment for this symbol and bar window.

The search engine is **pyne** (`POST /optimize` / `pynescript.optimize`). AXIS is the plugin UI. Pyodide and engines without `/optimize` fall back to an isolated local loop (random only).

This is not a live trading optimiser and not a Pine builtin. Best-on-sample is not a guarantee out of sample.

Optimise tab: search configuration and Fast/Slow input space

Holdout is the default for a reason. In-sample-only search overfits. Winners on one symbol or timeframe often fail on another.

Open it

```
Open a script that declares `strategy()`. Load history. A first **Run** is optional but helps engir

Results drawer → **Optimise**, or command palette **Optimise strategy**. Indicators and libraries s

Tick the `input.int` / `float` / `bool` / `enum` fields to search. Numeric fields need **min** and

Set trials, sampler, objective, and validation. **Start** runs the study. **Cancel** aborts the in-

**Apply best** merges the winner into Script Inputs (other inputs stay). **Apply + re-run** also ru
```

Study controls

Control	Default	Notes
Trials	30	Hard cap 200
Sampler	auto	Random if N<20, else TPE (TPE needs pyne `/optimize`)
Objective	composite	Also net PnL, profit factor, Calmar-like
Validation	holdout 30%	Walk-forward or in-sample (warned)
Min trades	5	Fewer closed trades → trial rejected

Grid is offered when the cartesian product fits the trial budget. On Pyodide (no `/optimize`) the UI falls back to random and says so.

Validation

Mode	What happens
Holdout (default)	Sampler sees the first ~70% of bars. Every trial is also scored on the last ~30%. Ranking prefers out-of-sample.
Walk-forward	Rolling train/test windows. Engine runs ≈ `N × folds` (capped at 400).
In-sample	One run on all bars. Warned in the UI.

Train and test are **bar slices**, not trade filters on a full-sample run.

Engines

Engine	Path
`server` (Pro API)	`POST {endpoint}/optimize` — parse-cached interpret loop, TPE / random / grid
Pyodide / missing `/optimize`	Isolated `runScript` — random only

Same-origin VPS / Docker must **proxy `/optimize`** to Flask (same as `/run`). Otherwise AXIS falls back to the slow client loop.

Overfitting

- Prefer holdout. Treat in-sample net profit as a search score, not a live forecast.
- Composite + min-trades rejects lottery-ticket parameter sets with 1–2 fills.
- Apply + re-run on a later date range or another symbol before trusting a winner.

See also

- Strategy and results
- pyne POST /optimize
- pyne CLI `optimize`

DRAWINGS

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Drawings" description: "Interactive chart annotations in AXIS—tools, persistence, and how they differ from Pine script drawings."

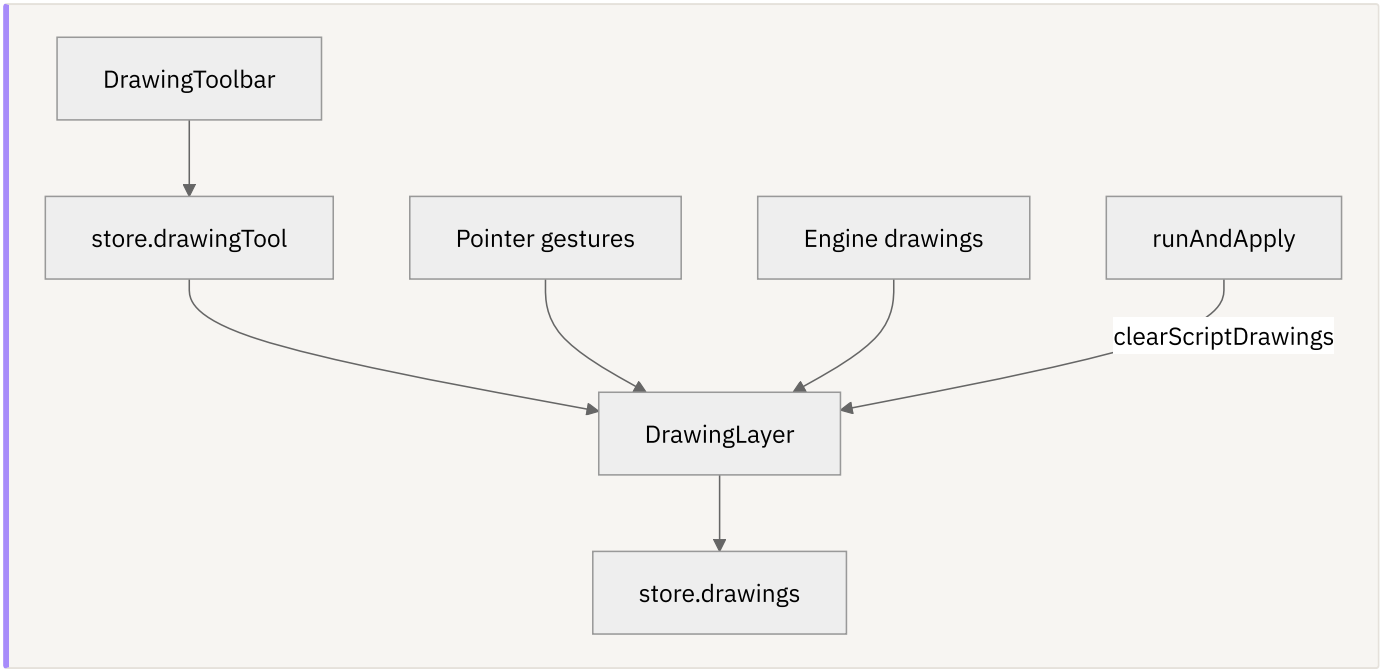
DRAWINGS

AXIS supports **user drawings** (interactive tools on the chart) and **script drawings** (objects returned by the engine). They share a canvas layer but different lifecycles.

Abstract

Class	Origin	Cleared when	Persisted
User drawings	Drawing toolbar	Explicit delete / clear	Yes (`store.drawings` in `pynescript.axis.v1`)
Script drawings	Engine `drawings[]`	Next Run (re-applied)	No (recomputed)

Conceptual model



Fibonacci retracement drawn on BTCUSDT with the drawing toolbar Placing a trend line on the AXIS chart

Tools

Left drawing tool rail: cursor, trend, shapes, fib, measure

Tool id	Kind	Geometry
`cursor`	—	Select / pan mode (not a drawable)
`hline`	hline	Horizontal price
`vline`	vline	Vertical time
`trend`	trend	Two points
`ray`	ray	Two points (extends)
`extend`	extend	Extended line
`rect`	rect	Two corners
`ellipse`	ellipse	Ellipse / oval
`arrow`	arrow	Directional segment
`fib`	fib	Two points; levels 0, 0.236, 0.382, 0.5, 0.618, 0.786, 1
`measure`	measure	Two points (delta readout)
`text`	text	Anchor + label

Colors: default accent `#939fff`, up/down/measure variants in `DRAWING_COLORS`.

Style bar and per-tool settings

The floating style bar (next to the rail) edits **the selected drawing**, or the **defaults for the active tool** when nothing is selected. Last-used extras are stored per tool (`drawingPrefs.byKind`).

Control	Where	Tools
Color / width / dash	Style bar	All place tools
Custom color	Style bar native picker	All place tools
Fill	Style bar slider	Shapes, ranges, fib, highlighter, positions
Extend L / R	Style bar arrows	Trend, ray, extend, info line, channel, fib
Show price	Style bar `$`	Horizontal/vertical lines, fib labels
Lock	Style bar (selection)	Selected drawing
Text / font size	Settings popover (gear); double-click on chart	Text, note, callout, flag, price label
Fib levels / reverse / %	Settings popover	Fibonacci family
Risk/reward	Settings popover	Long / short
Arrow caps	Settings popover	Trend, ray, arrow, forecast

Gear button: `axis-drawing-settings`. Popover: `axis-drawing-settings-popover`.

Layers & templates

- **Layers** panel lists user drawings (select, hide, delete)
- **Templates**: save/load drawing packs (`axis.drawingTemplates.v1`)
- **Duplicate / tag symbol** helpers for multi-chart workflows

Interface surface

- Toolbar on chart host (`DrawingToolbar`)
- Active tool in store; layer sync via `createEffect` on `store.drawingTool`
- Pane manager hosts price chart; drawings bind to time/price coordinates (not pixel-only)

Workflows

Annotate a level

1. Select Horizontal line.
2. Click price.
3. Reload page—line remains (persisted drawings array).

Fibonacci between swing points

1. Select Fib.
2. Click swing low → swing high (or reverse).
3. Levels render from `FIB_LEVELS`.

After Run with Pine lines/labels

1. Run script that emits drawings.
2. Layer clears previous script drawings, then applies new set.
3. User drawings remain.

Script drawings (engine → chart)

Engine `drawings[]` map through `pyne-drawings.ts` before paint:

Engine surface	Chart behavior
`line` / `box` / `label` / `polyline`	SVG geometry on the script's pane scale
`linefill` / `line_fill`	Filled quad between two line endpoints
`force_overlay` (line/box/label)	Paint on the **price** pane even when the script is `overlay=false`
`barcolor` series	Not SVG — per-bar candle body/wick tint via `plot-visuals`
`table.*`	Chart overlay HUD by default, or the bottom **Tables** panel (see below)

Non-overlay scripts place ordinary geometry on the indicator pane Y-scale; user toolbar drawings always stay on price.

Pine tables: chart HUD vs bottom Tables panel

Pine Script™ `table.*` drawings render as a chart overlay HUD (`PyneTableHud`) by default. Any script's tables can move off the chart into a bottom **Tables** tab — TradingView® September 2026 parity:

1. Chart **⋯ menu** (or the pane badge ⋯) → **Move tables to bottom**.
2. Tables stack **vertically in creation order**, stretch to the pane width with proportional columns from `table.cell(width=…)`, keep tooltips and merged cells, never truncate text (scrollbars when needed), and allow text **selection / copy** — unlike chart overlay tables.
3. Move back via **Move tables to chart** (same menus).

Placement persists per script (`store.pineTablesLocation[scriptId]`: `chart` default / `bottom`). One tab per script; the chart HUD hides moved tables.

Internals

Path	Role
`src/chart/drawing-types.ts`	Tool ids, types, fib levels
`src/chart/drawing-layer.ts`	Interaction + render
`src/chart/DrawingToolbar.tsx`	UI
`src/chart/pyne-drawings.ts`	Map engine drawings → layer
`src/chart/ChartHost.tsx`	Mount / dispose

Invariants

1. User drawings ≠ strategy results; deleting drawings does not alter `lastRun`.
2. Time base is bar time (unix-compatible chart times).
3. `cursor` never creates geometry.
4. Persistence omits bars/logs but **includes** drawings—large freehand-less sets only.

Failure modes

Symptom	Fix
Clicks do nothing	Tool is cursor; chart empty (no bars/time scale)
Drawings vanish on Run	Those were script drawings—re-run or convert notes to user tools
Lost after wipe	Cleared site data / different browser profile

See also

- AXIS — charting
- Strategy and results

ALERTS

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Alerts" description: "AXIS local alerts: price crosses, drawings, Pine conditions, on-chain spikes. Persist axis.alerts.v1. Not HOOX orders."

ALERTS

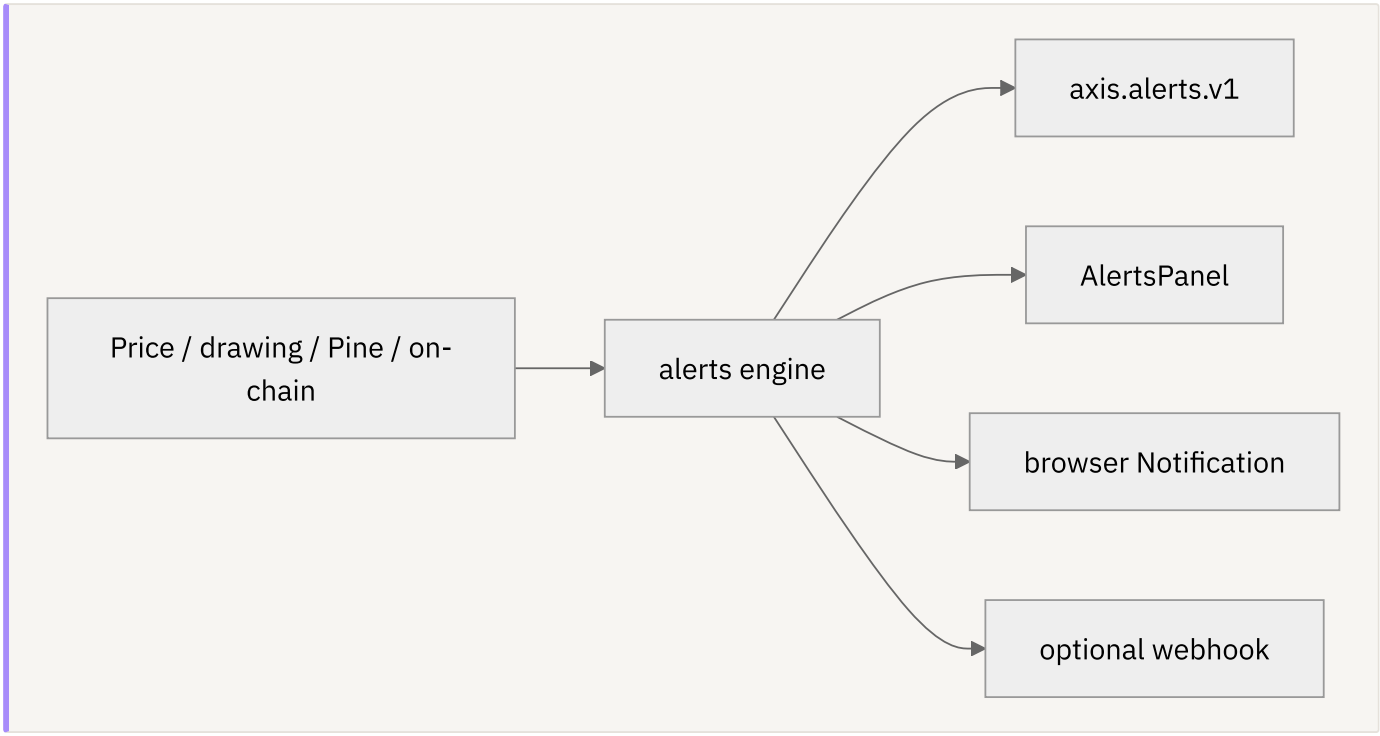
Abstract

AXIS ships a **client-side alert book** (`src/alerts/`) with an Alerts panel. Kinds cover price, percent change, drawing touch, indicator plot conditions, and on-chain TVL spikes. State persists in `axis.alerts.v1`. Delivery is local to the PWA: browser notifications, an optional HTTPS webhook, and an optional **L2 webhook** (richer JSON: `channel: "l2"`, message, params snapshot).

These alerts are **not** HOOX `trade-worker` orders. Pointing the L2 field at a PYNE/HOOX gateway is possible and **dangerous** — you are then inventing an execution path the PWA does not certify. Pine **`alert()`** and **`alertcondition()`** (and strategy `alert_message` on fills) fire as `pine_alert` rows when you arm them in the panel. Interpret runs export `alerts` / `alert_conditions`; AXIS delivers only the **last bar** so a full backtest does not dump history. Plot-vs-threshold remains `pine_condition`.

Alerts panel: price-cross form and empty book

Conceptual model



Interface surface

Kinds implemented in `src/alerts/`:

Kind	Fires when
`price_cross`	Price crosses a level
`price_above` / `price_below`	Price vs level
`pct_change`	Bar-to-bar (or window) percent move
`drawing_touch`	Price touches a user drawing's levels (hline, fib, trend anchors)
`pine_condition`	Applied indicator plot vs threshold (`>` , `<` , `cross` , …)
`pine_alert`	Pine `alert()` / `alertcondition()` (and strategy `alert_message`) on the last bar
`onchain_tvl_spike`	DefiLlama TVL jump (see On-Chain)
`onchain_event`	Catalogued on-chain event

UI: `src/ui/AlertsPanel.tsx` . Persistence key: **`axis.alerts.v1`** .

Delivery: enable **Notifications** in the panel (browser permission), optionally set a compact webhook URL, and/or an L2 webhook. Both URLs must be `https` (no loopback / private IPs). Live ticks from the chart stream evaluate price, drawing, and indicator kinds; on-chain kinds still fire from the on-chain events plane.

PYNE language `alert()` / `alertcondition()` on Flask / pyne-worker is documented at PYNE alerts.

Internals

Path	Role
`src/alerts/`	Engine + format + storage
`src/ui/AlertsPanel.tsx`	Panel
`src/onchain/`	TVL spike helpers (`buildTvlSpikeEvents`)

Invariants & edge cases

1. Alerts survive reload via local storage, not the Worker D1 script registry.
2. Closing the tab stops evaluation unless a stream + engine is still running.
3. On-chain kinds require the on-chain proxy (`/api/onchain`) to be reachable.
4. `lastRun` strategy results are **not** persisted; do not assume alert history equals a backtest.

Worked examples

1. Open Alerts panel.
2. Add `price_cross` on the active symbol / TF.
3. Enable browser notifications in the browser, then tick **Browser notification** on the form.
4. For an indicator: apply a script, pick its plot, set `>` / `<` / `cross` vs a threshold.
5. For a drawing: place a horizontal line (or fib), pick it in the form, optionally set a tolerance.
6. For Pine `alert()` / `alertcondition()` : run a strategy, open Alerts, kind **pine alert**, pick the script and `alert()` / `alertcondition()` / both. Optional title filter from the last run. In-browser compile records these in the compiled loop; the Flask/pyne-worker engine uses interpret for scripts that call them.

On-chain: attach a TVL series, then add `onchain_tvl_spike` (default ~10% day-over-day in `buildTvlSpikeEvents`).

Failure modes

Symptom	Cause	Fix
No rows	Engine not running / no stream	Start live or replay bars
Pine `alert()` never fires	No `pine_alert` row armed; or remote compile without AXIS fallback	Arm **pine alert**; AXIS uses interpret on Flask/pyne-worker when the script calls `alert()` / `alertcondition()` . In-browser compile records them in the compiled loop.
Pine kind never fires	Engine dropped alerts	Use Flask interpret; see PYNE alerts
On-chain silent	Proxy HTML / CORS	Use Worker `/api/onchain` , not the SPA origin
Webhook posted a trade	You pointed it at HOOX	Use pyne-worker instead

See also

- On-Chain

- Drawings
- Evaluation map
- PYNE alerts
- AXIS and HOOX

MCP

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "MCP (agents)" description: "Connect Claude, Cursor, Grok, or Inspector to AXIS so an agent can load symbols, edit Pine, run scripts, and read results." sidebarTitle: "MCP"

MCP (AGENTS)

Abstract

AXIS speaks **Model Context Protocol** so a coding agent can **read and write** the product: load a symbol, edit the Pine buffer, run a script, inspect strategy results, toggle panels, manage alerts and the library.

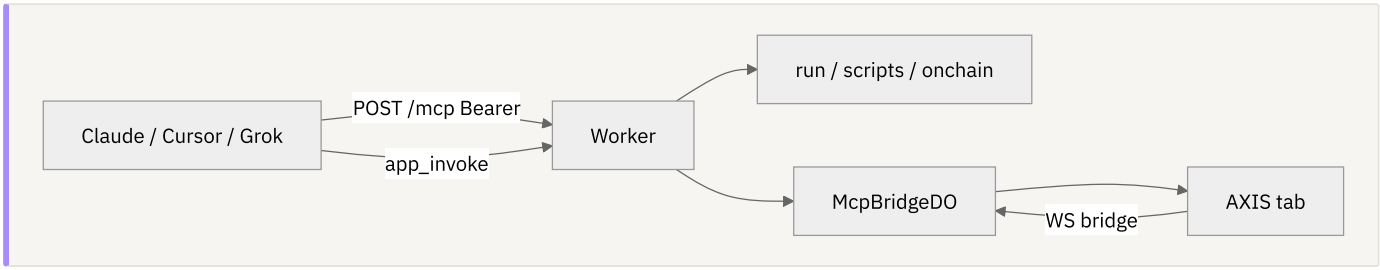
Two planes:

Plane	Live AXIS tab?	Typical use
Worker	No	Health, `POST /api/run`, cloud scripts, on-chain / market proxy
App	Yes — this browser session	Chart, editor, indicators, alerts, watchlist, library, results, chrome

Auth is a Worker API key (`pn_…`), the same Bearer used for cloud script storage. Secrets (exchange credentials, API keys) are redacted from snapshots.

Operator contract and tool schemas: MCP server.

Conceptual model



Connect this tab

1. Mint a key: `axis keys create` (needs `ADMIN_TOKEN`).
2. **Studio → Settings → General → Worker (cloud + MCP)** — paste Worker URL (`https://worker.axis.hoox.sh` or `http://127.0.0.1:8787`) and the `pn_…` key. Data tab is exchange credentials only.
3. **Studio → Settings → General → MCP** — leave **Connect this tab to MCP** on (default).
4. Confirm the hint shows `Bridge: open` (or `connecting`). `idle` with "No Worker API key" means step 2 is missing.

The PWA mints a short-lived one-time ticket over HTTP (`Authorization: Bearer`) then opens `wss://…/api/mcp/bridge?ticket=…` so the long-lived `pn_…` key never appears in the WebSocket URL. The Worker partitions the session by a hash of the key — another user's key cannot attach to your tab.

Turn the toggle **off** if you do not want agents driving this browser.

Status-bar indicator

The status bar shows an **MCP capsule** with two dots:

- **Session dot** — bridge socket state (green open, amber connecting, red error/off, dim idle) plus how many PWA tabs the Worker sees on your key's session (`MCP · 2 tabs`). The count polls every 20 seconds while connected.
- **Activity dot** — lights up while an agent is actively driving this tab (any `app_invoke` received in the last few seconds). Hover for the last capability and when it ran.

Click the capsule to open Settings → MCP.

Note: MCP clients (Claude, Cursor, …) call `POST /mcp` statelessly, so there is no per-client presence — the capsule tracks bridge + tab attachment and live invoke activity, which is what `app_*` tools need.

MCP in System Logs

Every bridge event lands in **System Logs** (topbar **Logs**) with source `mcp`: connects, disconnects, session tab-count changes, and one line per agent invoke (`drawings.add → ok · 84ms`, errors with their code). Filter the strip by source (`mcp`) or level, or query programmatically with `logs.get` (`{ source?, level?, limit? }`).

New app versions prompt via an update banner (version check every 2 minutes, on window focus, and when the bridge reconnects) — no manual reload hunting.

Connect a client

Print a snippet:

```
axis mcp config --key pn_…
```

Endpoint: `https://worker.axis.hoox.sh/mcp` (or your wrangler origin + `/mcp`).

Remote HTTP (Inspector, Cursor, clients with URL + headers)

```
{
  "mcpServers": {
    "axis": {
      "url": "https://worker.axis.hoox.sh/mcp",
      "headers": { "Authorization": "Bearer pn_…" }
    }
  }
}
```

`GET /mcp` is public discovery (tool names only). Tool calls require Bearer.

Stdio (`axis mcp`)

For clients that only spawn a local process:

```
axis mcp --key pn_…
# or AXIS_API_KEY=pn_… axis mcp
axis mcp config --stdio --key pn_…
```

The CLI forwards newline-delimited JSON-RPC from stdin to `POST /mcp`.

What to ask an agent

Worker-only (no tab):

- “Call `axis_health` and tell me if D1 and MCP are on.”
- “List my cloud scripts.” (`axis_scripts_list`)
- “Run this Pine against these OHLCV rows.” (`axis_run`)

Live tab (`app_session_status` → connected):

- “Load BTCUSDT 1h and tell me bar count and last close.” (`chart.load` then `app_get` `bars`)
- “Put this indicator in the editor and run it.” (`editor.set` + `editor.run`)
- “Open the alerts panel and list alerts.” (`panels.set` + `alerts.list`)
- “Screenshot the panes.” (`chart.screenshot` — PNG base64 of the Lightweight Charts panes)
- “My chart seems a bar behind — what's wrong?” (`chart.lag` — the verdict compares the chart tail to the venue clock, the tick, and the engine series)
- “Where are the holes in my bars?” (`chart.bars` — bounded tail/head rows + a missing-slot map)
- “Is the engine's series advancing, or is the socket wedged?” (`stream.get` + `engine.series`)
- “Tick the engine now / repair the stale series / reset the socket.” (`engine.tick` · `engine.repair` · `stream.reset` — the last one drops the socket and subscriptions so the next tick re-subscribes) Start with

`app_capabilities` if the client has not cached the catalog.

Capabilities at a glance

Pass these ids to **`app_invoke`** as `capability`, with a JSON `payload`.

Domain	Ids
App	`app.snapshot`, `app.get`, `app.set`, `app.command`, `app.shortcut`, `app.event`
Chart	`chart.get` / `set` / `load` / `reload` / `live` / `layout` / `theme` / `type` / `screenshot` / `zoom` / `bars` / `lag`
Editor	`editor.get` / `set` / `run` / `diagnostics`
Scripts	`indicators.list` / `add` / `remove` / `update` / `run` (remove detaches chart panes like the Layers panel; code edits need `run` after)
Alerts	`alerts.list` / `create` / `update` / `remove`
Watchlist	`watchlist.get` / `add` / `remove` / `lists`
Library	`library.list` / `read` / `write` / `remove`
Results / logs	`results.get`, `logs.get` / `append` / `clear`
Chrome	`panels.get` / `set`, `theme.get` / `set`, `plugins.list` / `activate`
Workspace	`workspace.export` / `import`
Drawings	`drawings.list` / `add` / `update` / `remove` / `clear` / `tool`
Settings	`settings.get` / `set`
Engine	`engine.get` / `series` (read the engine's own series bounds + staleness) · `engine.tick` / `repair` (force a drive tick / stale-series repair)
Stream	`stream.get` (route, engine socket, subscriptions, feed chips) · `stream.reset` (drop socket + subscriptions, probes a re-subscribe)
Status	`status.get`

`app_set` paths (anything else is `PATH_DENIED`): `symbol`, `interval`, `source`, `engine`, `exchange`, `theme`, `chartType`, `uiScale`, `live.active`, `live.preferAfterLoad`, `editor.doc`, `editor.open`, `watchlist.open`.

Example payloads:

```
{ "capability": "chart.load", "payload": { "symbol": "ETHUSDT", "interval": "15m" } }
```

```
{ "capability": "editor.set", "payload": { "doc": "//@version=6\nindicator(\"RSI\")\nplot(ta.rsi(cl
```

```
{ "capability": "alerts.create", "payload": { "name": "BTC 100k", "kind": "price_cross", "symbol":
```

```
{ "capability": "drawings.add", "payload": { "kind": "trend", "points": [{ "time": 1700000000, "pri
```

```
{ "capability": "settings.set", "payload": { "settings": { "interval": "1h", "historyBars": 5000 }
```

```
{ "capability": "chart.lag", "payload": {} }
```

```
{ "capability": "chart.bars", "payload": { "limit": 20, "gaps": true } }
```

```
{ "capability": "engine.tick", "payload": {} }
```

```
{ "capability": "stream.reset", "payload": { "resubscribe": true } }
```

Full argument tables: MCP server.

Limits

- **No tab** → Worker tools still work; `app_*` returns `SESSION_OFFLINE`.
- **`MCP_BRIDGE` unbound** → `NO_MCP_BRIDGE` (add the Durable Object in `wrangler.toml` and deploy).
- **`axis_request` is not an open proxy** — only AXIS JSON APIs. Stream, OAuth device-flow, and `/mcp` recursion are blocked.
- Agents must not invent Pine APIs (`TV.set` , `pine.chart.setTheme` , …). See AGENTS.md in the repo.
- MCP is **not** HOOX order routing. Alerts remain local (browser / webhook).

Troubleshooting

Symptom	Check
401 `NO_KEY` / `INVALID_KEY`	Bearer on `/mcp` ; mint with `axis keys create`
503 `API_KEYS_REQUIRED`	D1 bound without `API_KEYS` KV — `axis setup kv`
`SESSION_OFFLINE`	Tab open, MCP toggle on, same `pn_…` as the client
`Bridge: idle` / no API key	Settings → General → Worker (cloud + MCP)
`Bridge: error`	Worker URL reachable; CORS; `MCP_BRIDGE` deployed
App command unknown	Palette has not mounted yet — use `chart.*` / `editor.*` ids instead

In-page test (browser console on the AXIS origin):

```
await window.__AXIS_MCP__.invoke('chart.get')
```

See also

- MCP server — tools, resources, prompts, allowlist
- AXIS CLI
- Auth
- Manager and settings

SCREENSHOTS

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED) SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Interface gallery" description: "Stills and looping GIFs of the AXIS PWA, Studio, and CLI — captured from the public demo."

INTERFACE GALLERY

Product chrome as shipped. Stills live under `docs/images/`. GIFs are short looping tours, not live-exchange recordings.

Capture: `bun run capture:1080p` (Playwright, **1920×1080 @2x** → 3840×2160). A second series at **2560×1440 @2x** (5120×2880) lives under `docs/images/2560x1440/`.

AXIS workspace with BTCUSDT daily candles, watchlist, volume, RSI pane, and Pine editor

Workspace

Load BTCUSDT history then run an RSI indicator

Still	What
workspace.png	Default layout: watchlist, chart, editor
chart-panes.png	Price + volume + RSI panes
topbar.png	Market / data / compute / panel groups
watchlist.png	Symbol rail
statusbar.png	Connection HUD
command-palette.png	⌘K / Ctrl+K
layers.png	Panes, overlays, scripts, drawings
theme-panel.png	Settings → Theme presets
workspace-mobile.png	390×844 PWA

Switching chart theme presets from Settings

Chart & drawings

Fibonacci retracement on BTCUSDT with RSI pane

Still	What
drawings-toolbar.png	Left tool rail
drawings-fib.png	Fibonacci retracement
bar-replay.png	Replay strip on the chart
compare.png	Second-symbol compare
volume-profile.png	Volume profile overlay
chart-layout-2x2.png	2×2 multi-chart grid

Placing a trend line on the chart Bar replay scrub and play over loaded BTCUSDT history

Editor & scripts

CodeMirror 6 Pine editor with a v6 RSI script

Still	What
editor.png	Docked Pine editor
scripts-panel.png	Applied scripts
script-library.png	Local library
scriptlogs.png	Script logs
run-rsi.png	Workspace after an RSI run

Research panels

Still	What
results-drawer.png	Strategy results overlay
results-tabs.png	Events / Strategy / Optimise / …
results-strategy.png	Closed trades + equity
hpo.png	Hyperparameter search
data-source-manager.png	Background backfill
onchain-tvl.png	DefiLlama TVL overlay
onchain-panel.png	On-Chain panel (TVL)
onchain-dex.png	DEX pool OHLCV tab
alerts.png	Local alerts

Attaching Aave TVL from the On-Chain panel

Studio

AXIS Studio Runtime page with server engine and pynescript.online endpoint

Switching Studio pages: Runtime, Wire, Settings, Workers, Plugins

Still	What
runtime.png	Active calculation
wire.png	Compose recipes
settings.png	Appearance / live / keys
workers.png	Backend inventory
plugins.png	Contract catalog
plugins-install.png	Install from URL

CLI

axis --help in a dark terminal

Still	What
help.png	`axis --help`
doctor.png	`axis doctor`

Landing crops

Finish-cut tiles used on hoox.sh/axis: `docs/images/landing/`.

Landing hero: AXIS workspace with BTCUSDT and RSI

Tile	File
Chart panes	axis-chart.png
Drawings	axis-drawings.png
Editor	axis-editor.png
On-chain	axis-onchain.png
Results	axis-results.png
Studio Wire	axis-studio.png
Mobile	axis-mobile.png
Open Graph	axis-og.png

TROUBLESHOOTING

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Troubleshooting" description: "Diagnose empty charts, CORS, engine failures, Pyodide assets, streams, storage, and PWA cache issues in AXIS."

TROUBLESHOOTING

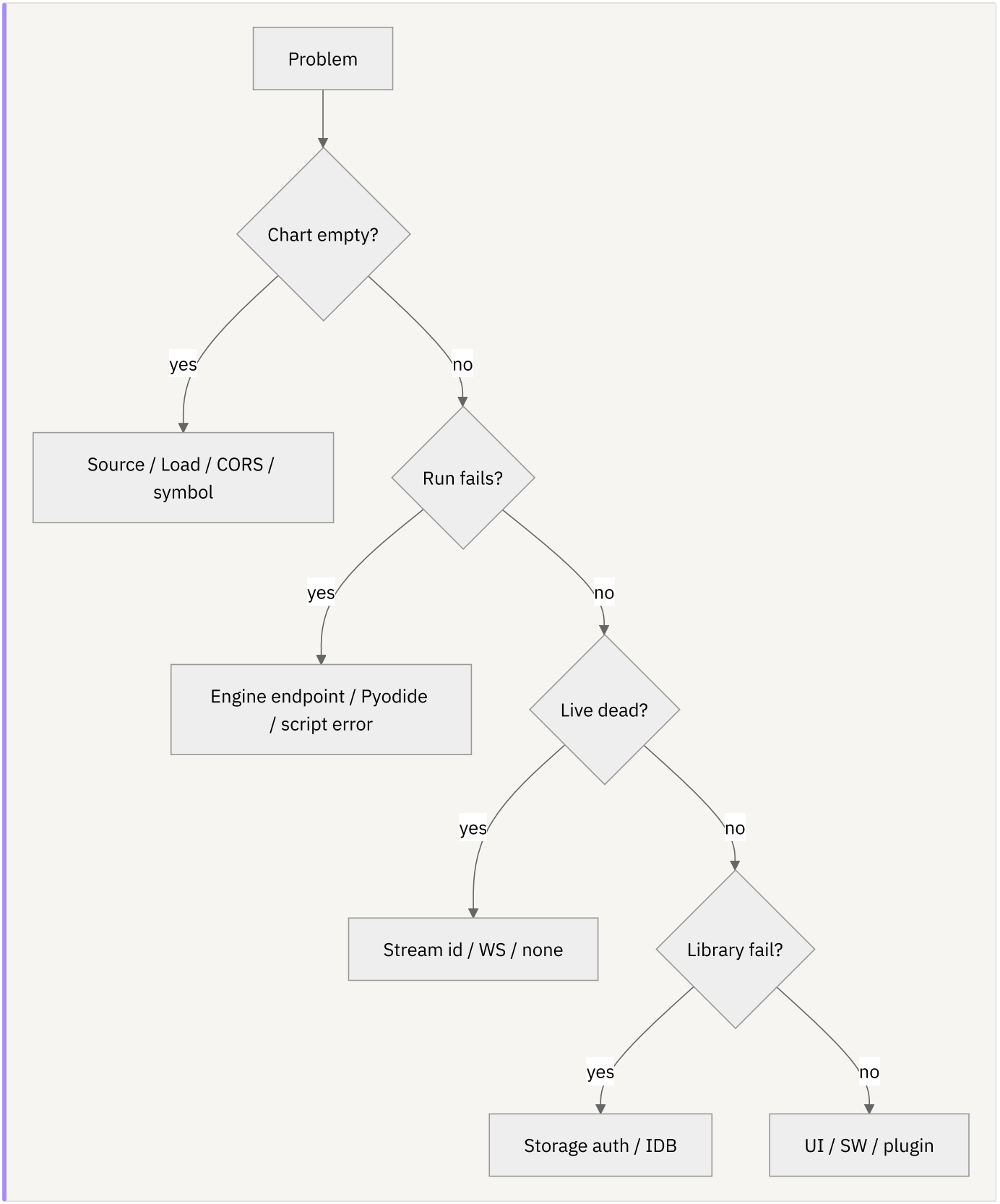
Systematic triage for AXIS. Prefer logs drawer + Network tab + status bar before reinstalling.

Abstract

Failure domains:

1. **Data plane** — source / stream
2. **Calc plane** — engine / endpoint / Pyodide assets
3. **Shell plane** — store, SW cache, plugins
4. **Storage plane** — library backends
5. **MCP plane** — `/mcp` auth, PWA bridge

Decision tree



Data plane

Symptom	Checks	Fix
Infinite loading	Network klines; status message	Correct symbol; try `mock-walk`
CORS on REST	Binance/public APIs usually OK; custom sources may fail	Proxy plugin or Worker
CSV no bars	Parse error in status	Fix columns/time; re-upload
Wrong interval bars	Store interval vs request	Settings interval + Load

Calc plane — server engine

Symptom	Checks	Fix
Failed fetch `/run`	Endpoint, CORS preflight	`ALLOWED_ORIGINS`; Settings probe
HTTP 4xx/5xx	Response JSON `message`	Fix script; backend logs
Timeout	Bar count × complexity	Shorter history; raise server resources
Plots missing	`status: success` but empty series	Script has no plots; check Raw

Probe: Settings → Test endpoint (health GET).

Calc plane — Pyodide

Symptom	Checks	Fix
HTML instead of wheel	Content-Type / ZIP magic	Deploy `public/vendor`, `public/pyodide` into `dist/`
Slow first run	Cold runtime	Wait; `preloadPyodide` on idle
Offline fail first visit	Assets never cached	One online warm-up
Interpreter error	Results Raw / logs	Language issue → PYNE

Live streams

Symptom	Checks	Fix
Live toggle no effect	Live is off (`stream: null`)	Select `binance-ws` or `mock-poll`
WS errors	Console; venue symbol format	Uppercase `BTCUSDT` style
Reconnect loop	Network flap	Logs drawer; stop/start Live
DO stream	Worker session query	Worker docs; example plugin

Storage / library

Symptom	Checks	Fix
Save no-op	Active storage; errors in panel	Read status line
Cloud 401	Bearer key	Settings/library key field
Git denied	Token scopes	PAT permissions
Lost local scripts	Cleared site data	Import JSON backup

Chart / editor (recent surfaces)

Symptom	Checks	Fix
Indicator blank / missing on chart	Oscillator on price scale; wrong pane id	Prefer `overlay=false`; re-run; see Indicators
Editor no lines / zero height	Dock height chain	Detach/reattach or toggle panel; editor should fill right strip
Editor covers Indicators	Side-by-side dock	Both open on right → Indicators left of Editor; resize each panel
Replay shows one candle only	Session start	Replay starts at **last bar** with full history; Play at end restarts from bar 0
No Debug chips / Pins	Toggles + log shape	Enable **Debug** / **Pins**; logs need `line` and/or `bar_index` — Debugging
Completions thin	Builtins corpus	Re-sync `pyne-builtins.json` from PYNE; optional remote LSP
Pre-eval stalls for seconds on long scripts	Idle call-site scan	**2.27.0** is single-pass O(n); hard-reload if an older SW still serves the old editor bundle
Missing `W002` / `W104` offline	Settings → Editor toggles	Enable **Warn if //@version is missing** and **Warn on v6-removed kwargs**; works without Pro API
Go to definition / rename no-op	Focus + symbol under cursor	`F12` / `Shift-F12` / `F2` (or command palette); builtins / keywords refuse rename
Pan/zoom hitch under live overlays	Full-series rewrite mid-gesture	**2.26.0** tip-only ranging defers `setData` / markers until the gesture ends — hard-reload if still on an older build
Pyodide wheel HTML / BadZipFile	`/vendor/pynescript-*.whl`	Confirm `dist/` has the shipped wheel (`pynescript-0.6.29` on **2.27.0**) — not an SPA HTML fallback

Shell / PWA

Symptom	Checks	Fix
Old UI after deploy	Service Worker	Unregister SW; hard reload
State weird	`localStorage['pynescript.axis.v1']`	Export scripts; clear key; reload
Plugin vanished	URL install list	Re-install URL
Theme wrong	`data-theme`	Toggle theme once
Lost multi-chart slots	`chartLayout` in store	Named layouts menu; re-apply recipe

MCP

Symptom	Checks	Fix
401 on `/mcp`	Bearer `pn_…`	`axis keys create`; client `Authorization` header
`SESSION_OFFLINE`	Tab open? Same key?	Settings → MCP on; Settings → General → Worker (cloud + MCP) has the key
`NO_MCP_BRIDGE`	Worker health `features.mcpBridge`	Bind `MCP_BRIDGE` in wrangler.toml and deploy
Bridge idle	Cloud API key	Settings → General → Worker (cloud + MCP)

Guide: MCP (agents).

Logging

- **System logs** drawer: boot, pyodide ready, live re-run errors (`appendLog`)
- **Status bar**: ready / loading / running / error + short message
- **Results** → **Raw**: last engine payload

Security-related

- Poisoned localStorage: app should tolerate parse failures (see security tests)
- Plugin URL schemes restricted
- Never paste production PATs into shared screen recordings

Internals for deeper digs

Area	Path
Load history	`data/load-symbol.ts`
Run pipeline	`indicators/runner.ts`
Streams	`streams/multiplex.ts`
Engines	`engines/catalog.ts`
Store	`store/index.ts`
MCP host	`src/mcp/`
MCP Worker	`worker/src/mcp/`

See also

- Installation
- FAQ
- DevOps
- Worker
- MCP (agents)

GLOSSARY

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Glossary" description: "AXIS end-user glossary: axes, AXIS, plugins, engines, storage keys, and result terms."

GLOSSARY

Terms as used in AXIS documentation and UI. Language runtime terms → PYNE.

Product

Term	Definition
AXIS	Charting PWA product : chart, editor, panels, and store—not the language engine.
UI track	Docs track for presentation surfaces (chart host, editor, shell, store) under `/axis/docs/ui`.
PYNE	Pine Script™ toolchain (parse/eval); engines embed or call it.
Active set	Tuple `{ source, stream, engine, storage }` currently selected.
Compose / recipe	A purposeful active set for a workflow (offline lab, desk, edge, …).
MCP	Model Context Protocol. AXIS Worker `POST /mcp` plus an optional PWA bridge so agents can read/write the app.

Plugin kinds

Term	Definition
Source	Historical OHLCV provider (`fetchHistorical`).
Stream	Live bar/tick push (`start` → dispose).
Engine	Calculator (`isReady` , `run`).
Storage	Script library backend (`list` / `read` / `write` / `remove`).
Component	Reserved UI slot plugins (phase 2).
Registry	In-memory catalog of plugins by kind (`PluginRegistry`).
configSchema	Declarative field schema for Settings / plugin config UI.
Capabilities	Flags: offline, needsAuth, needsNetwork, needsProxy.

Engines & backends

Term	Definition
server engine	`POST {endpoint}/run` with script + bars.
pyodide engine	In-browser Python + pynescript wheels (product **2.27.0** vendors `pynescript-0.6.29`).
Pro API / Flask	Local Python backend (`make run` , typically `:5002`).
Worker	Cloudflare® Worker data plane for AXIS.
Endpoint	Base URL the server engine (and often cloud storage) calls.

Data & chart

Term	Definition
Bar	`{ time, open, high, low, close, volume? }`
Pane	Chart strip: price, volume, indicator, equity.
Overlay	Plot drawn on price pane (`meta.overlay !== false`).
Sub-pane	Non-overlay indicator strip; stable store id `indicator` .
User drawing	Interactive annotation tool geometry.
Script drawing	Engine-emitted line/label/box objects.
Marker	Trade/event marker on series.
Pane badge	Corner chip with script actions (settings / load source / eye / re-run / remove).
Multi-chart	Grid layouts (1 / 2H / 2V / 4) with per-slot symbol/interval.
Bar Replay	Scrub/play over loaded OHLCV without live stream.
Compare	Second-symbol series overlaid on the price pane.
Volume profile	OHLCV volume-at-price histogram (Layers).
Watchlist	Side panel of symbols + quote refresh.
Scripts panel	Applied indicators/strategies list (UI title **Scripts**; internal id `indicators`).
Layers	Panel listing panes, scripts, and user drawings.
Data Source Manager	Background multi-page OHLCV backfill to a past date + gap validation (`Data` panel).
Bars cache	IndexedDB OHLCV store keyed by source/symbol/interval for the manager.
Chart theme preset	Named token set (void, classic, mono, obsidian, ...) for chart + chrome accents.
Alert	Local price condition (+ optional webhook); Alerts panel.

Editor & debug

Term	Definition
Debug	Inline end-of-line chips from last-run logs with line refs.
Pins	Chart markers + 📍 gutter for logs with bar_index/time.
Problems	Clickable list of engine diagnostics for the last run.
Ruler	80-character column guide in the Pine editor.
Symbols	Editor catalog of TV-editor-safe arrows, box drawing, and chart emoji.
Command palette	⌘/Ctrl+K jump menu for panels, Run, toggles, git.
Git bar	Pull/Push via active storage plugin (local/cloud/git).

Results

Term	Definition
lastRun	In-memory last engine payload.
Event	Strategy/order/plot-adjacent event from engine.
Closed trade	Paired entry/exit with PnL in the viewer.
Equity curve	Cumulative PnL series derived from trades/events.
Scriptlogs	Floatable panel of Pine `log.*` from last run (not System Logs).
Optimise	Results tab that searches `strategy()` `input.*` values over N trials (pyne `/optimize` or isolated client fallback).
HPO	Hyperparameter optimisation — not a Pine builtin.

Persistence

Term	Definition
`pynescript.axis.v1`	Primary localStorage app state key.
`pynescript.axis.v2`	Older app-state key; migrated on read.
pluginsConfig	Per-plugin configuration map.
Library export	JSON dump of storage documents.

Infrastructure names

Term	Definition
`axis`	CF Pages project id. Canonical URL `https://axis.hoox.sh`.
`axis.hoox.sh`	Canonical AXIS Pages URL.
`worker-axis`	CF Worker script name and health JSON `service`.
`worker.axis.hoox.sh`	Canonical AXIS Worker URL.
void	Brand pack / dark chrome aesthetic for AXIS docs/UI.

See also

- FAQ
- State namespaces
- Architecture overview

FAQ

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "FAQ" description: "Frequently asked questions about AXIS, engines, offline use, TradingView® relation, and data privacy."

FAQ

Product scope

Is AXIS TradingView®?

No. AXIS is an independent charting PWA. Pine Script™ and TradingView® are trademarks of TradingView®, Inc. This project is not affiliated with or endorsed by TradingView®, Inc. Trademark notice appears on the AXIS docs home.

Does AXIS implement Pine Script™?

Evaluation is performed by **engines** that call **PYNE** (or proxies that do). The AXIS does not embed a closed interpreter. See PYNE for language fidelity.

AXIS vs engine?

AXIS = UI + orchestration. Engine = `run(script, bars)`. You can swap engines without rewriting the chart host. That is **ADR-002** territory—ADRs.

Running & offline

Can I use AXIS fully offline?

Yes, with recipe **Offline lab**: `mock-walk` + `mock-poll` + `pyodide` + `local`, after one warm-up so Pyodide assets cache. Server engine and live venue streams require network.

Why is first Pyodide run slow?

Bootstraps Python + wheels in-browser. Subsequent runs reuse the runtime when still warm. Assets are self-hosted under the PWA origin when deployed correctly. Product **2.27.0** vendors `pynescript-0.6.29` under `/vendor/` (product PYNE may be ahead of the wheel).

Default endpoint points at a VPS—is that required?

No. Change Settings → Backend URL to `http://localhost:5002` or your Worker. Defaults may reference a public demo API host for convenience. AXIS CLI **0.3.6** defaults the Pro API to `https://pynescript.online` (`AXIS_PYNE_URL` overrides).

Data & privacy

Can an agent control AXIS?

Yes. The Worker serves MCP at `POST /mcp` (Bearer `pn_…`). Worker tools (`axis_run`, scripts, on-chain) do not need a tab. Chart/editor/panels need **Settings** → **MCP** connected. Guide: MCP (agents).

Where do API keys live?

In **this browser's** localStorage state / `pluginsConfig`. They are sent as HTTP headers only to endpoints you configure (cloud storage, MCP bridge, etc.). They are not part of the Pine script body. Snapshots returned to MCP clients redact secret-like fields.

Are OHLCV bars uploaded?

Server engine sends script + bars to the configured endpoint for that run. **Pyodide** keeps bars local. Choose engine accordingly.

Are bars saved on disk?

App state persistence **omits** `bars`, `lastRun`, and `logs` to control size. History is re-fetched on Load.

Features

How do I share a setup?

Legacy hash state (`state-hash.js`) can encode symbol/interval/engine/source/stream (and short scripts). Prefer documenting the four-tuple + library export for durable sharing. URL-load plugins are re-fetched by install list.

Can I use my own data vendor?

Yes—implement a **source** plugin (and optional stream) and install via URL or built-in registration. See Plugins.

Why Strategy tab shows no trades?

Need paired entry/exit-style events with prices. Indicators that only `plot` will fill Plots/Metrics, not Strategy stats.

Drawings disappeared after Run

Script drawings are cleared and re-applied each run. User toolbar drawings persist across runs.

Popout editor out of sync?

Editor bridge uses shared document storage + message bridge between windows. Use **Reattach** on the main AXIS if the popout closed uncleanly.

What is Debug vs Pins?

	Debug	Pins
Shows	End-of-line chips in the editor	Markers on the chart + 📍 gutter
Needs	Line reference in logs/errors	`bar_index` and/or bar time
Toggle	Editor header **Debug**	**Pins** or **Alt+P**

Full workflow: Debugging.

Does the editor work offline (definition, rename, lint)?

Yes for local intel. **2.27.0** adds Pine symbol navigation (`F12` / `Shift-F12` / `F2`) and offline PYNE lint parity for `W002` (pre-v5 `//@version`) and `W104` (removed `when=` / `transp=` kwargs). Toggles live under Settings → Editor. Pre-eval call scanning is O(n) so long scripts stay responsive while typing.

Multi-chart, Compare, Replay, Alerts?

- **Layouts** topbar menu: 1 / 2H / 2V / 4 + named recipes
- **Compare**: second symbol overlay (% or absolute)
- **Replay**: scrub loaded history (starts with full history at last bar)
- **Alerts**: local price alerts panel (webhook optional)
- **⌘/Ctrl+K**: command palette (panels, Run, editor toggles, git)

UI detail: UI shell, Charting.

Why are Scripts next to the Editor?

Left/right docks with multiple open panels lay out **side-by-side** (row), so the **Scripts** panel can sit left of Editor on the right strip. Bottom dock still stacks. (Internal panel id is still `indicators` for saved layouts.)

Can AXIS tune strategy inputs?

Yes — Results → **Optimise**, or command palette **Optimise strategy**. Strategies only. pyne `POST /optimize` runs TPE / random / grid with holdout by default; Pyodide falls back to a local random loop. See Hyperparameter Optimisation.

How do I download years of history?

Topbar **Load** is one venue page. Open the **Data** panel (Data Source Manager) to backfill in the background to a past date, validate completeness, and fill gaps — see Data Source Manager.

Ops

What is `pynescript-axis`?

Former Cloudflare® Pages / Worker id. Pages project is now **`axis`** (`https://axis.hoox.sh`); Worker script is **`worker-axis`** (`https://worker.axis.hoox.sh`). Brand is AXIS.

Legacy localStorage keys?

Migrated automatically into `pynescript.axis.v1` and library keys. See State namespaces.

Tests for the PWA?

```
bun run test:unit
bun run test:e2e:smoke
```

See `TESTING.md`.

See also

- Troubleshooting
- Glossary
- Compose recipes